Reinforcement Learning

CS 59300: RL1

October 16, 2025

Joseph Campbell

Department of Computer Science

PURDUE UNIVERSITY®

Guest talk on Tuesday: Woojun Kim

Woojun Kim

Title: Toward Coordinated Intelligence for the Real World

Abstract: The real world is inherently multi-agent, consisting of numerous entities that must interact and coordinate in complex, dynamic environments. Achieving effective operation in such settings requires coordinated intelligence, where agents not only coordinate effectively but also address practical challenges including scalability, robustness, and safety. This talk will present methods for enhancing coordination and tackling these challenges in the context of multi-agent deep reinforcement learning, and will showcase applications to robotics, particularly focusing on robotic information gathering.

Homework 2 Questions?

Due next Tuesday 11:59 PM

CS 593 RL1 Fall 2025
Assignment 2: Deep Q Networks
Purdue University

Due October 21 2025, 11:59 PM

Installation

Setting up a Conda Environment

1. Open a terminal/command prompt and create a new environment:

```
conda create -n cs593rl python=3.9 -y
```

2. Activate the environment:

```
conda activate cs593rl
```

3. Navigate to the directory where you unzipped the code and install the necessary Python packages from `requirements.txt`:

```
pip install -r requirements.txt
```

This assigmnent uses two Gymnasium environments: LunarLander-v2 and CarRacing-v2. Refer to the documentation pages linked above for information on the state space, action space, and reward function (though in this assignment rewards are only reported as part of the evaluation).

Part 1: Implement DQN and Replay Buffer Variations [4 pts]

We have provided a starter implementation of DQN, with some functionality unimplemented. You have the option of running the code either on your own machine, on a cloud based service (such as Google Colab), or on RCAC Scholar.
Use the concepts introduced in class along with the Gymnasium/PyTorch documentation to fill in the blanks in the code marked with `TODO`.

1

Project proposals due Oct 30

2-page research-style paper outlining your project and progress

IEEE-style format

Template:
https://ras.papercept.net/conferences/support/support.php

Midterm November 4

Will release “high-level” study guide
In class at regular time and location

Topics:

- Bandits/MDPs
- Bellman equations, policy/value iteration
- DQNs
- Actor-Critic
- Multi-Agent RL
- Exploration

Today's lecture

1. Exploration in deep reinforcement learning

2. Intrinsic motivation

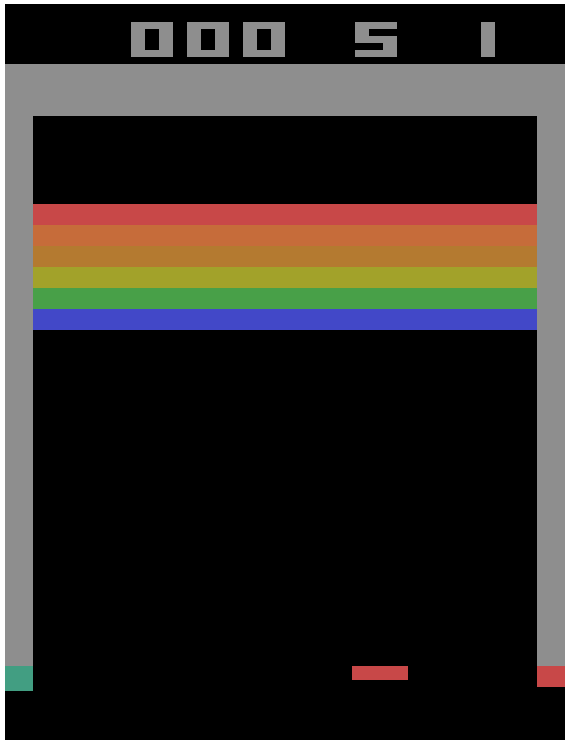
3. Extrinsic motivation (transfer learning)

Some content inspired by Sergey Levine's Berkeley CS 285 and Katerina Fragkiadaki's CMU 10-403

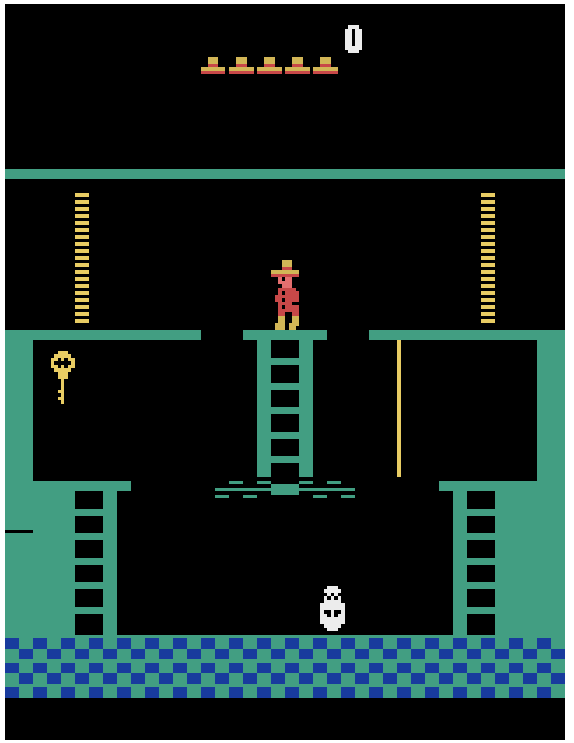
Exploration in deep reinforcement learning

Some problems are easy, some are hard…

Why is one game easy, and the other game is hard?



Easy
(Breakout)



Hard
(Montezuma's Revenge)

Montezuma’s revenge

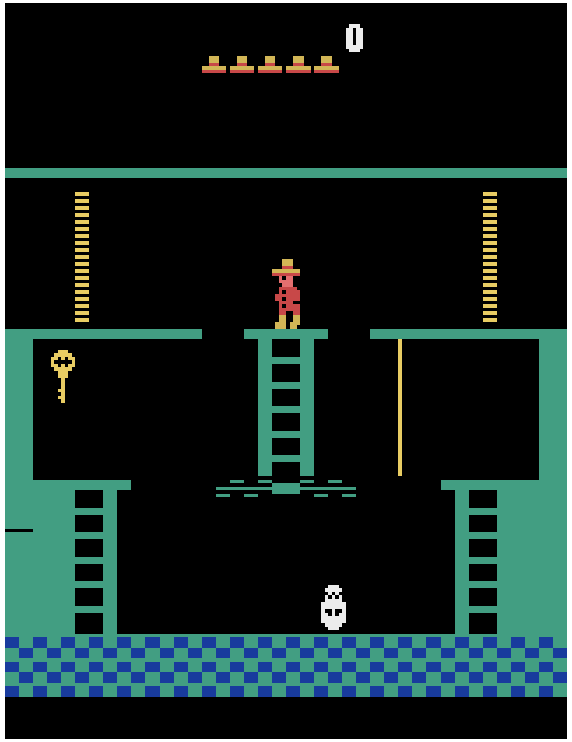
Positive reward for:

- Getting key
- Opening door

Getting killed gives no reward

Finishing the game is only weakly corelated with rewards

- **Why do we know how to beat it?**



What does the algorithm know?

Does not know the rules

- Can only discover them through trial-and-error

Rules don’t always make sense when discovered

- Observations may be less informative than what “we” see

Temporally extended tasks are difficult based on…

- …how extended the task is
- …how much you know about the rules

This is related to reward sparsity

Traditionally, we compensate for sparse rewards by reward shaping

Reward shaping:

- Add additional positive/negative rewards to make rewards “denser”
- If they are potential-based, then the optimal policy is the same as in the sparse reward case

Policy invariance under reward transformations: Theory and application to reward shaping

Andrew Y. Ng, Daishi Harada, Stuart Russell
Computer Science Division
University of California, Berkeley
Berkeley CA 94720
{ang,daishi,russell}@cs.berkeley.edu

Abstract

This paper investigates conditions under which modifications to the reward function of a Markov decision process preserve the optimal policy. It is shown that, besides the positive linear transformation familiar from utility theory, one can add a reward for transitions between states that is expressible as the difference in value of an *arbitrary* potential function applied to those states. Furthermore, this is shown to be a necessary condition for invariance, in the sense that any other transformation may yield suboptimal policies unless further assumptions are made about the underlying MDP. These results shed light on the practice of *reward shaping*, a method used in reinforcement learning whereby additional training rewards are used to guide the learning agent. In particular, some well-known “bugs” in reward shaping procedures are shown to arise from non-potential-based rewards, and methods are given for constructing shaping potentials corresponding to distance-based and subgoal-based heuristics. We show that such potentials can lead to substantial reductions in learning time.

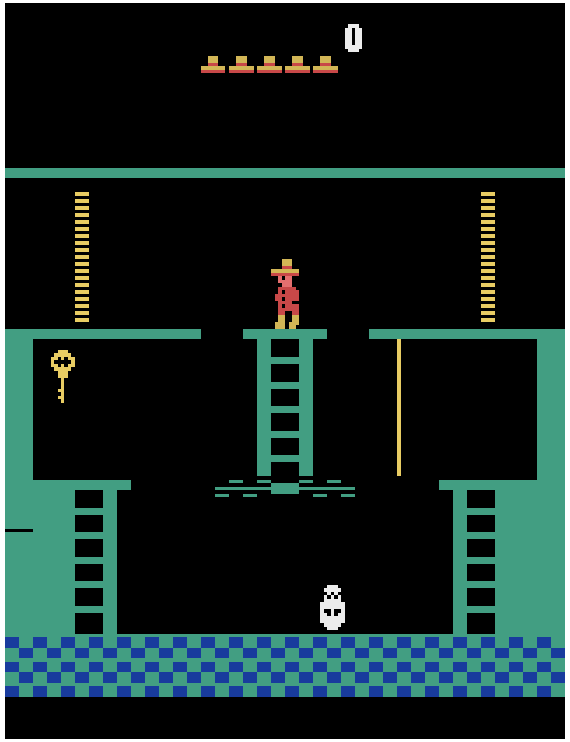
freedom do we have in specifying the reward function, such that the optimal policy remains unchanged?

In the field of utility theory, which studies primarily single-step decisions, the corresponding question for the utility function can be answered very simply. For single-step decisions without uncertainty, any monotonic transformation on utilities leaves the optimal decision unchanged; with uncertainty, only positive linear transformations are allowed [von Neumann and Morgenstern, 1944]. These results have important implications for designing evaluation functions in games, eliciting utility functions from humans, and many other areas.

To our knowledge, the question of policy invariance under reward function transformations has not been fully explored for sequential decision problems.[1] Policy-preserving transformations are important at least in these areas:

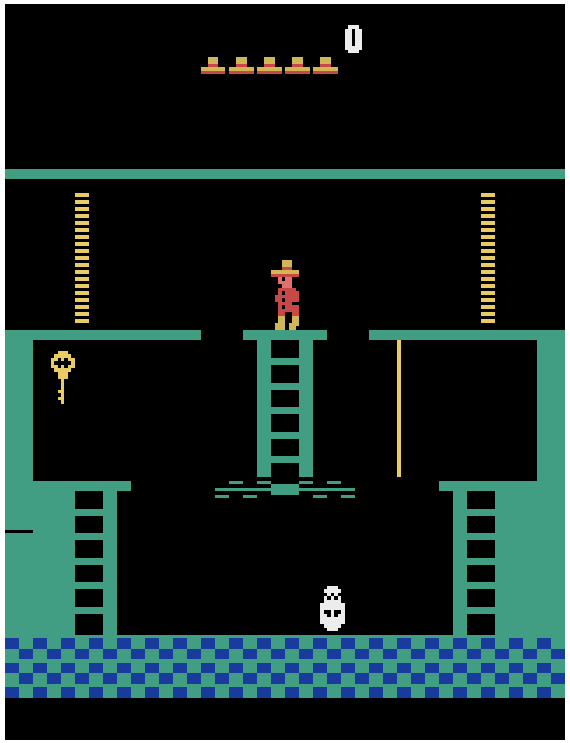
- The task of *structural estimation* of MDPs [Rust, 1994] involves recovering the model and reward function from observed optimal behavior. (See also the discussion of *inverse reinforcement learning* in [Russell, 1998].) Policy-preserving transformations determine the extent to which a reward function can be recovered.

But we can’t always shape rewards!



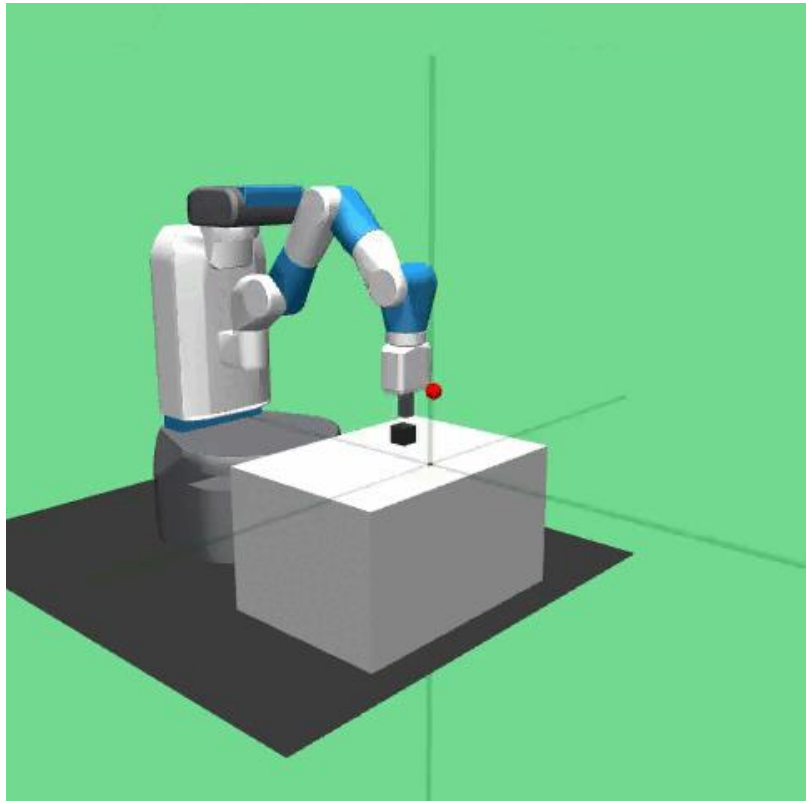
Why?

But we can’t always shape rewards!

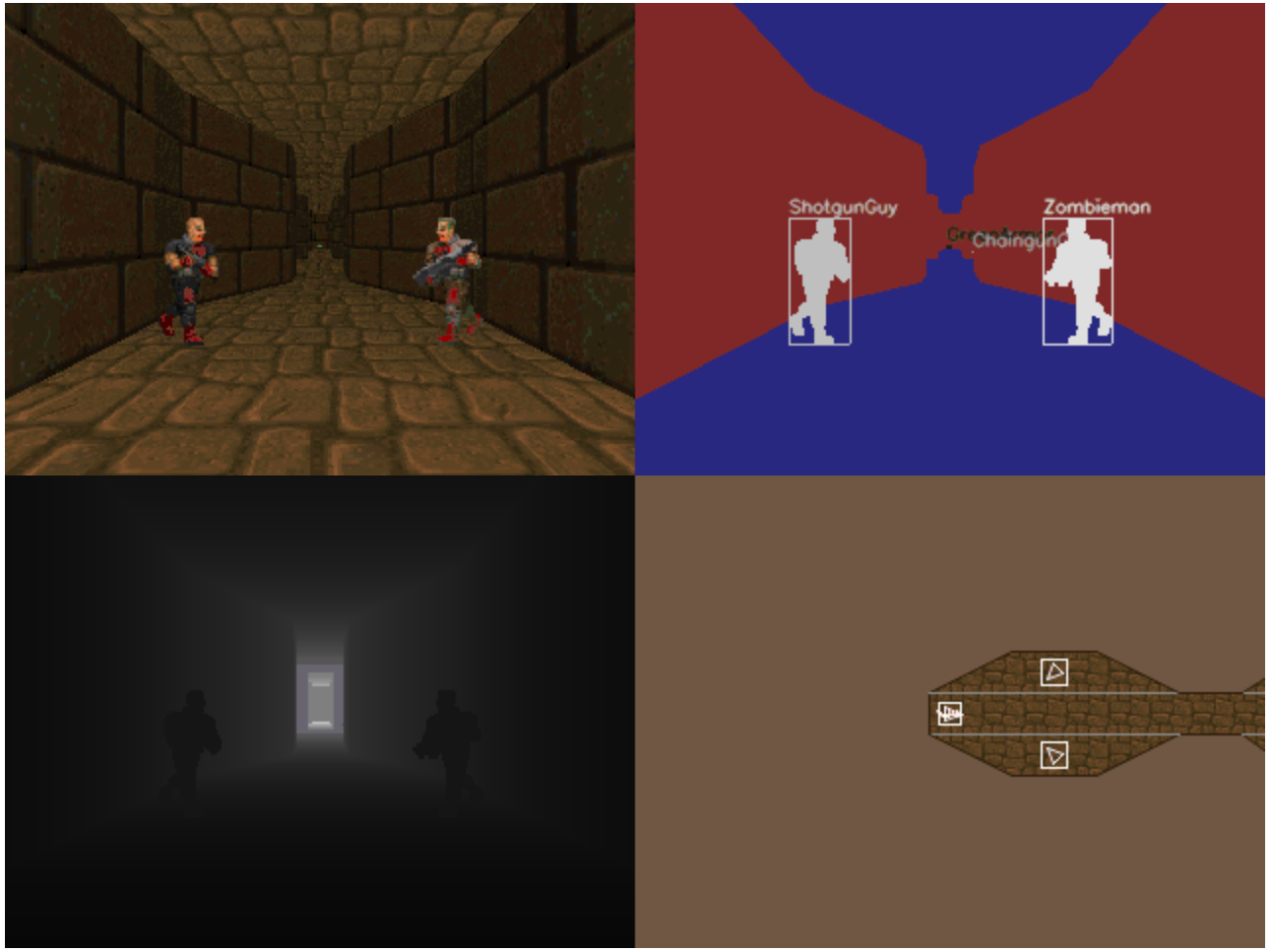


Our input is an image! How do we know when to issue rewards?

More examples



https://robotics.farama.org/envs/fetch/pick_and_place/



https://vizdoom.farama.org/index.html

Exploration-exploitation dilemma

Recall from earlier lectures…

- How can an agent discover high-reward strategies that require a temporally extended sequence of complex behaviors
  - May individually not be rewarding
- How can an agent decide whether to attempt new behaviors or continue with the best one it knows about so far

Exploitation: doing what you *know* will yield the highest reward

Exploration: doing new things in the hopes of finding higher rewards

Exploration is hard

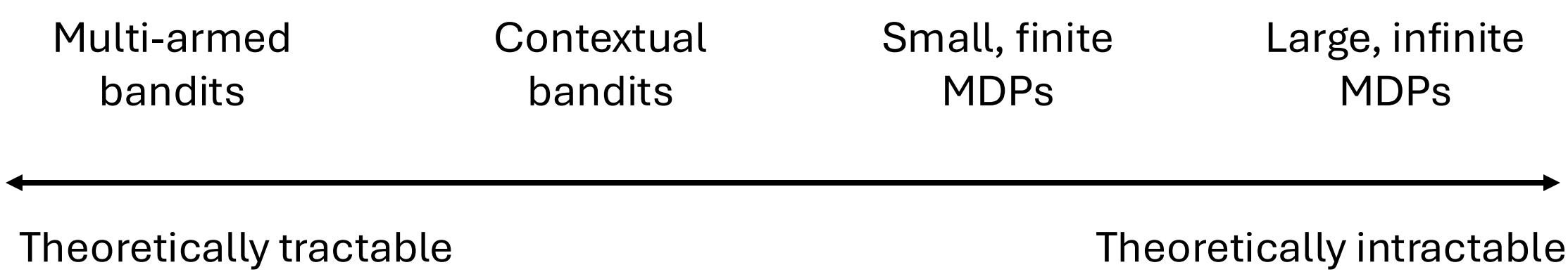
Can we derive an optimal exploration strategy?

Multi-armed bandits | Contextual bandits | Small, finite MDPs | Large, infinite MDPs

Theoretically tractable | Theoretically intractable

Exploration is hard

Can we derive an optimal exploration strategy?



In general: no

What’s wrong with previous methods?

What are previous methods we’ve discussed to tackle exploration?

What's wrong with previous methods?

What are previous methods we've discussed to tackle exploration?

1. ϵ-greedy

2. Upper confidence bounds (UCB)

Recap: ϵ-greedy

Idea: start by exploring a lot and gradually explore less over time as our reward estimates become more accurate.

This is ϵ-greedy!

With greedy action selection you always exploit.

With ϵ-greedy action selection you are usually greedy, but with some probability ϵ you take a random action

- Simplest way to balance exploration-exploitation!

ϵ-greedy is very inefficient!

ϵ-greedy results in uniform exploration across the state space

- Always behaving a *little* random no matter where the agent is
- This is not intelligent exploration, we may end up in states we've been in before

This means that ϵ-greedy is often ineffective in large MDPs

- ϵ too big? Don't sufficiently exploit
- ϵ too small? Don't sufficiently explore

Recap: Upper Confidence Bound in Bandits

Estimate an upper confidence $\widehat{U}_t(a)$ for each action-value such that with high probability:

$$Q(a) \leq \hat{Q}_t + \widehat{U}_t(a)$$

The confidence depends on the number of times that action a has been selected…

- Small number of times → a large uncertainty
- Large number of times → a small uncertainty

Upper Confidence Bound in MDP

$$a = \arg\max \underbrace{\hat{Q}_t}_{\text{Exploit}} + c \underbrace{\sqrt{\frac{\ln(t)}{N_t(s,a)}}}_{\text{Explore}}$$

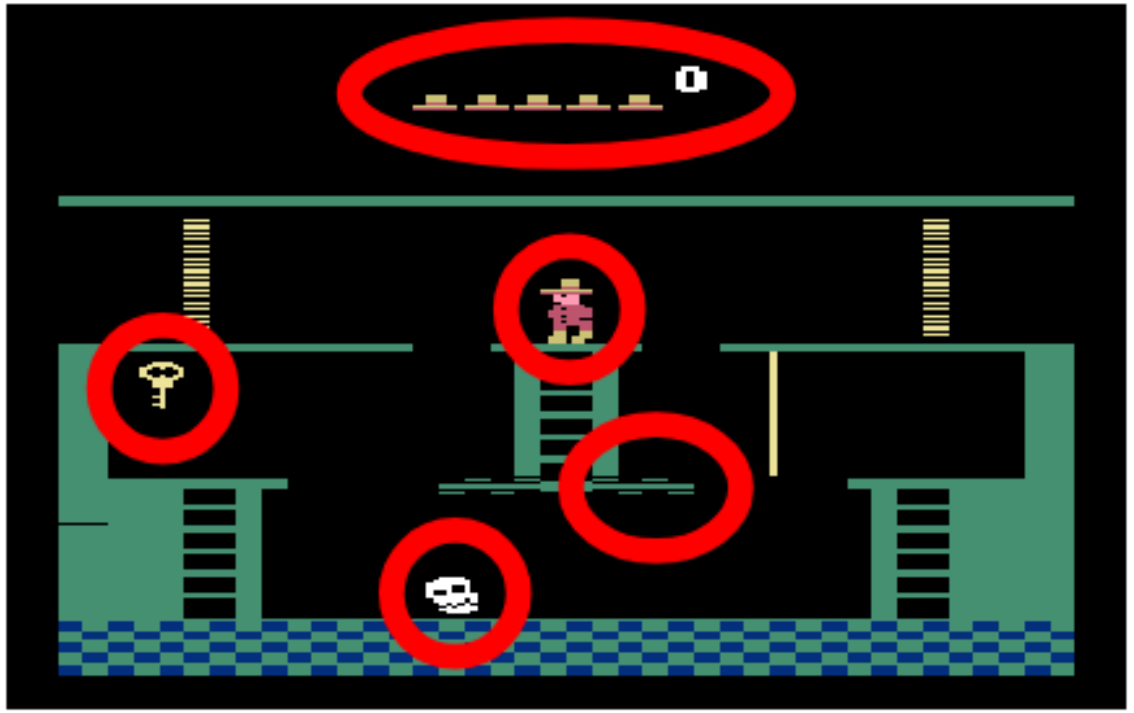
$N_t(s,a)$ is a count of how many times we've taken action a in state s

Why is this a problem?

What are we counting!?

We may never see the same thing twice!

- Although some states will be more similar than others
- How similar is similar-enough?



What exactly are we “counting”?

- Every possible state?

Intrinsic motivation

What is intrinsic motivation?

The agent provides its own rewards that encourage exploration!

- Reward is *not* given by the environment

Types of intrinsic motivation:

- **Novelty-seeking**: agent seeks out new experiences
- **Modeling error**: agent seeks to learn/model environment
- **Empowerment**: agent seeks states where it has control over env

Novelty-seeking methods

Idea: the agent seeks to explore states it has not previously seen

Rather than uniformly exploring as in ϵ-greedy, intelligently explore

- A random action might lead us to a state we have already been
- Instead…actively pursue novel states

Allows the agent to more efficiently explore and find high rewards

Pseudo-counting

Fit a density model (DNN) and use to obtain "pseudo-count"

- Unlike UCB, we are not counting states exactly

1. Fit $p_\theta(s)$ to all states $\mathcal{D}$ seen so far
2. At time t, take action and get s_{t+1}
3. Fit $p'_\theta(s)$ to $\mathcal{D} \cup s_{t+1}$
4. Use p_θ and p'_θ to estimate $\hat{N}(s_{t+1})$
5. Set $r_{t+1} = r_{t+1} + \beta\left(\hat{N}(s_{t+1})\right)^{-1}$

Bellemare et al, Unifying Count-Based Exploration and Intrinsic Motivation, 2016.

Pseudo-counting

The density models p_θ and p'_θ tell us how likely it is to encounter state s after a sequence of states $s_{1:n}$

- High density means the agent is likely to encounter state s

$$p_\theta(s) = \frac{\hat{N}(s)}{\hat{n}}, \qquad p'_\theta = \frac{\hat{N}(s)+1}{\hat{n}+1}$$

$$\hat{N}(s) = \frac{p_\theta(s)(1-p'_\theta(s))}{p'_\theta(s)-p_\theta(s)}$$

Intuition: if we have already visited s a lot, then p_θ should not change much after visiting it *one more time*

Pseudo-counting example

If we assume p_θ is tabular and represents count/total, then we can solve exactly for the total count based on the change in p_θ

$$\widehat{N}(s) = \frac{p_\theta(s)(1 - p'_\theta(s))}{p'_\theta(s) - p_\theta(s)}$$

Suppose $p_\theta(s) = 0.01$ and $p_\theta(s) = 0.02$

$\frac{0.01(1-0.02)}{0.02-0.01} = \frac{0.0098}{0.01} = 0.98$ which is roughly ~1 prior visit

Pseudo-counting example

If we assume p_θ is tabular and represents count/total, then we can solve exactly for the total count based on the change in p_θ

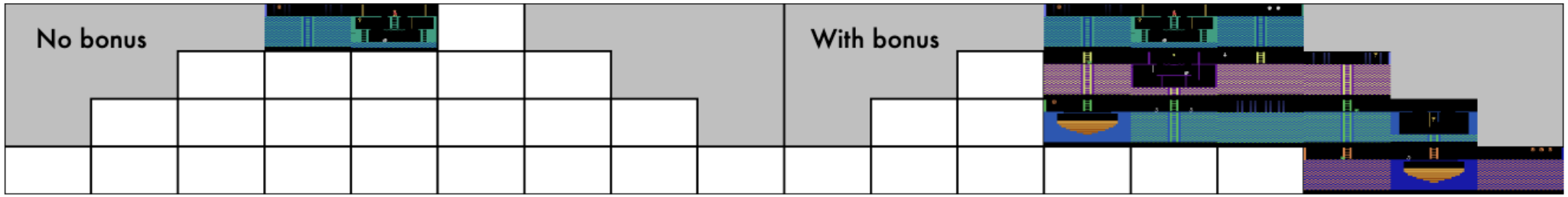
$$\widehat{N}(s) = \frac{p_\theta(s)(1 - p'_\theta(s))}{p'_\theta(s) - p_\theta(s)}$$

Suppose $p_\theta(s) = 0.1$ and $p_\theta(s) = 0.101$

$\frac{0.1(1-0.101)}{0.101-0.1} = \frac{0.0899}{0.001} = 89.9$ which is roughly 90 prior visits

Pseudo-counting

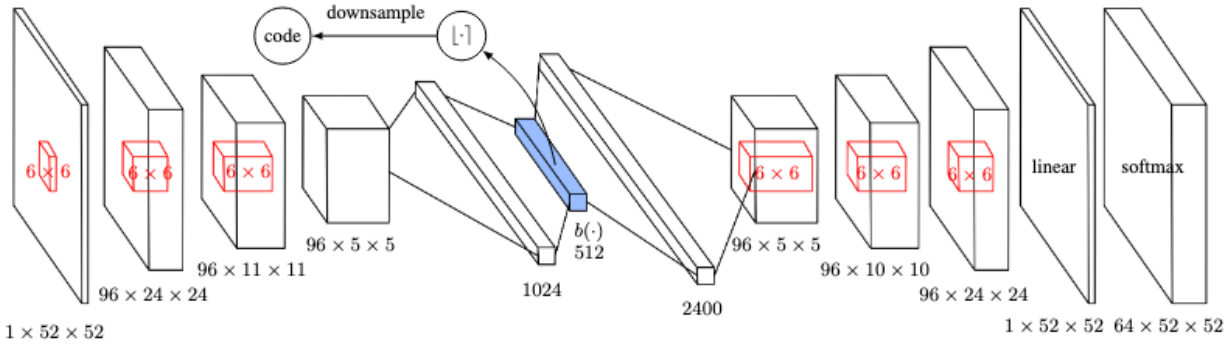
Works ok, but computationally expensive and density est. is difficult



Counting in a latent space

Instead of pseudo-counting, what if we explicitly count latent states

- Similar states are *hashed* to same code
- The hash code reflects the angular similarity of latent features
- Maintain visitation table over hash code counts



Tang et al, #Exploration: A Study of Count-Based Exploration for Deep Reinforcement Learning, 2017

Implicit densities using classification error

Instead of explicitly modeling densities, what if we try to distinguish the new state from all past states?

- State is novel if it is easy to distinguish from all previous states

Train a classifier to identify new state from all previous states

- Density = classifier error

Fu et al, EX2: Exploration with Exemplar Methods for Deep Reinforcement Learning, 2017

Implicit densities using classification error

$$D_{x^*}(x) = \frac{\delta_{x^*}(x)}{\delta_{x^*}(x) + P_{\mathcal{X}}(x)} \qquad \textit{and} \qquad D_{x^*}(x^*) = \frac{1}{1 + P_{\mathcal{X}}(x^*)}$$

$\delta_{x^*}(x)$ represents the point mass for an optimal discriminator

- For state x^*, $\delta_{x^*}(x) = 1$ when $x = x^*$ and 0 otherwise
- $P_{\mathcal{X}}$ is the state density over the dataset

Why does this work?

$$P_{\mathcal{X}}(x^*) = \frac{1 - D_{x^*}(x^*)}{D_{x^*}(x^*)}$$

Implicit densities using classification error

Algorithm 1 EX^2 for batch policy optimization

Initialize replay buffer B
for iteration i in $\{1, \ldots, \text{N}\}$ **do**
  Sample trajectories $\{\tau_j\}$ from policy π_i
  for state s in $\{\tau\}$ **do**
    Sample a batch of negatives $\{s'_k\}$ from B.
    Train discriminator D_s to minimize Eq. (1) with positive s, and negatives $\{s'_k\}$.
    Compute reward $R'(s, a) = R(s, a) + \beta f(D_s(s))$
  end for
  Improve π_i with respect to $R'(s, a)$ using any policy optimization method.
  $B \leftarrow B \cup \{\tau_i\}$
end for

Bootstrapped DQN

Q-value estimates are inherently uncertain, so use multiple Q-value models to represent this uncertainty

- Remember *Double Q-learning*?

During each episode, randomly select one Q-function model to follow

- Effectively sampling from different hypotheses about environment

Osband et al, Deep Exploration via Bootstrapped DQN, 2016

Modeling error methods

Model learning and model improvement can be cast as goal-seeking behavior

- Goal is not to beat Montezuma's revenge, but to improve model

Agent seeks out states that it can't model or predict

Implicit densities using state prediction error

Measure how “predictable” or “unpredictable” a given state is

- Uses a fixed randomly initialized target network
- Train a predictor network to predict the output of the target network for a given state

Exploration bonus is how well the predictor can approximate the output of the target network

Burda et al, Exploration by Random Network Distillation, 2018

Curiosity

The agent learns to predict the outcome of its actions

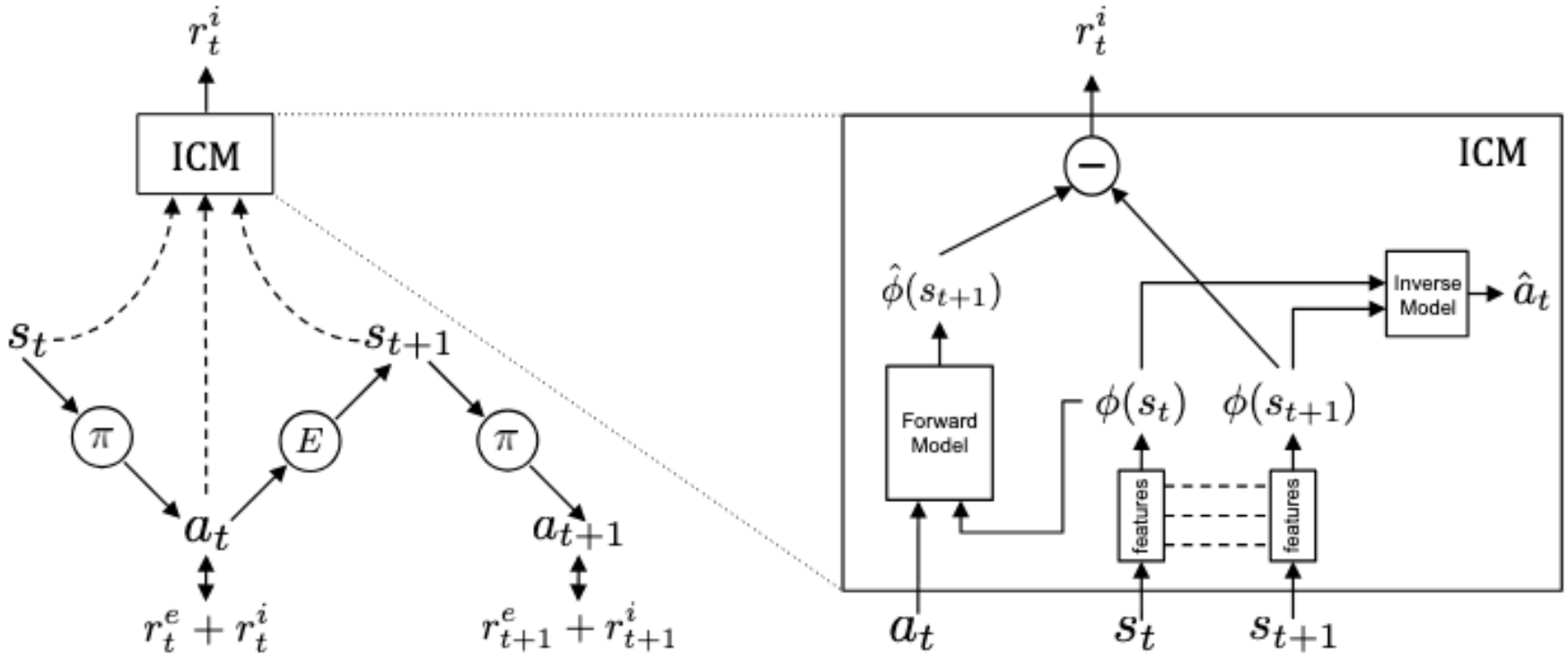
- Visual forward dynamics
- Self-supervised learning (no labels!)

Agent takes an action and compares its predicted outcome to actual

- The error is the exploration bonus

Pathak et al, Curiosity-Driven Exploration by Self-Supervised Prediction, 2017

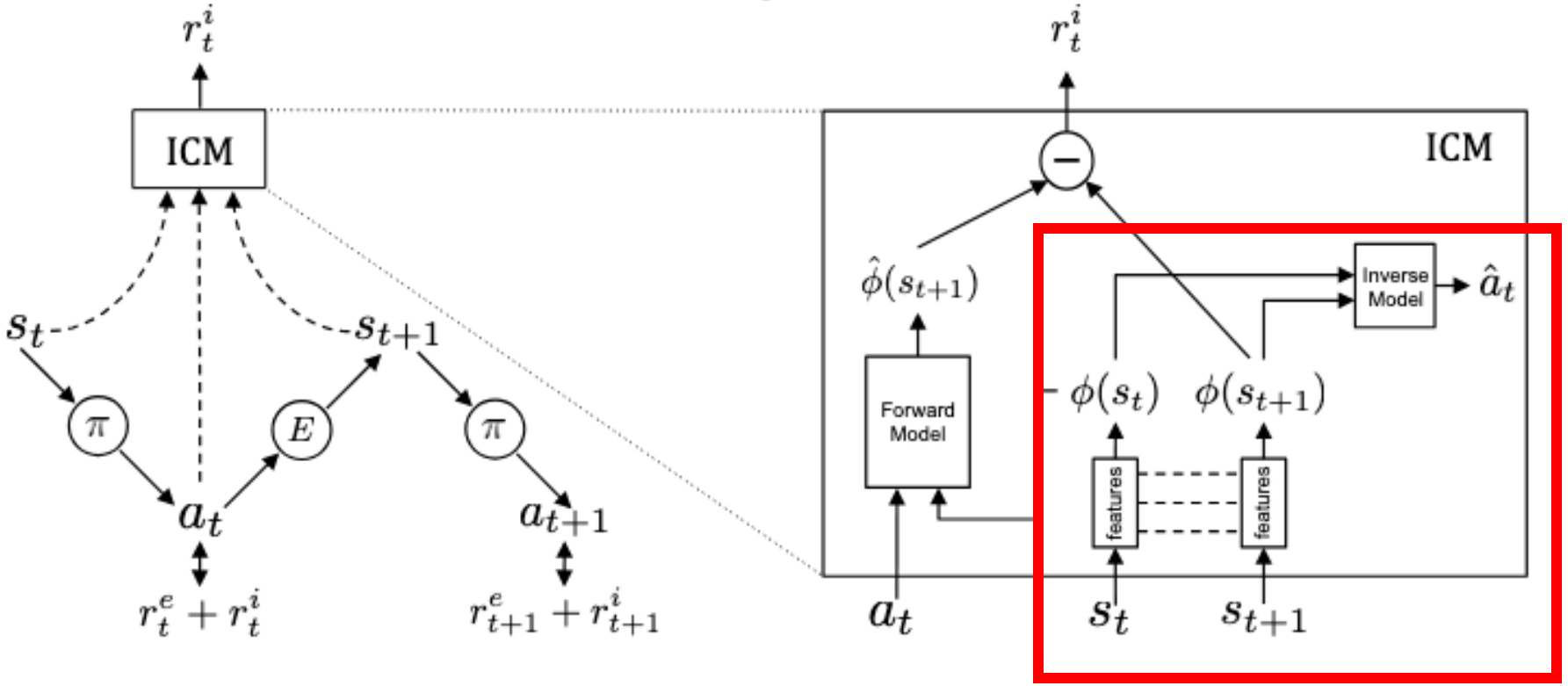
Intrinsic curiosity module



Inverse model used to train meaningful state representation (features)

- Goal: only model effect of agent's own actions!

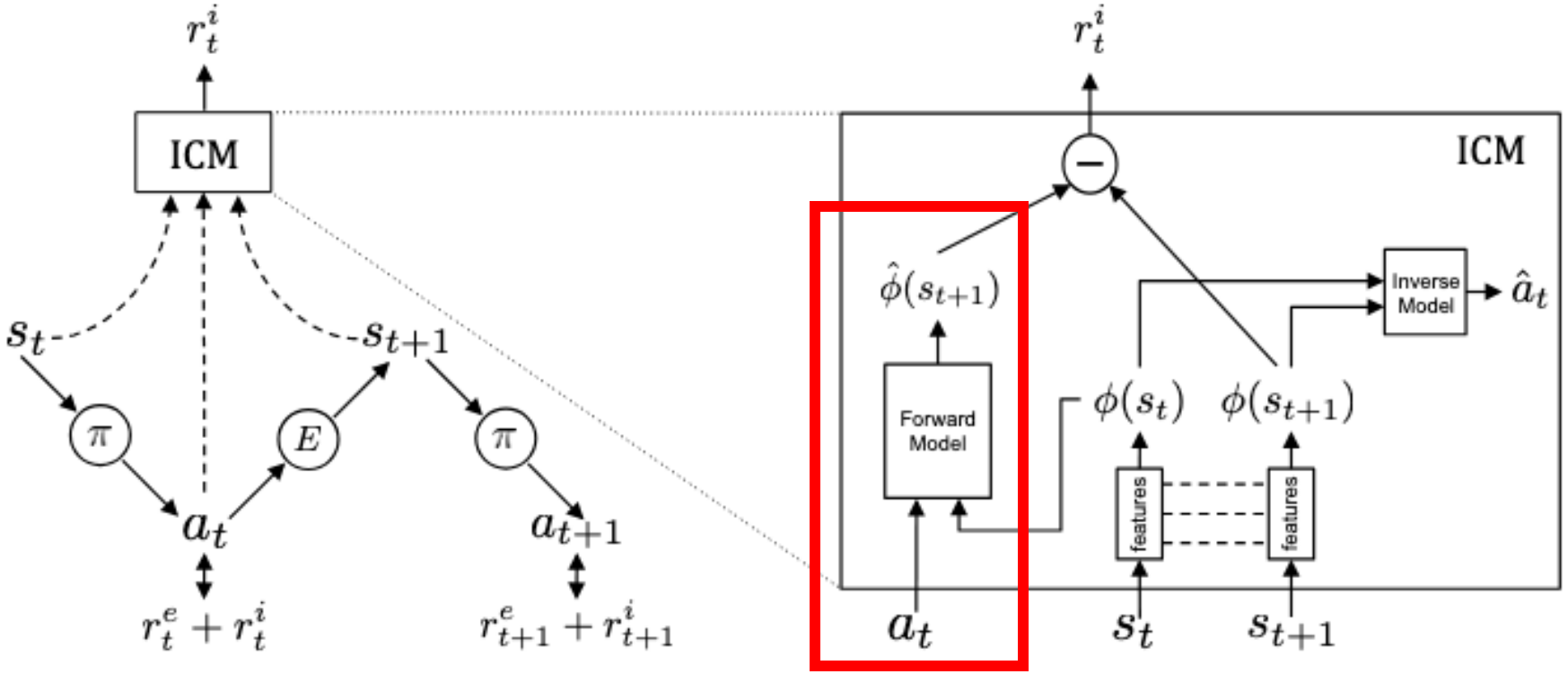
Only model effects of agent's actions



Predict what action was taken to go from $\phi(s_t)$ to $\phi(s_{t+1})$

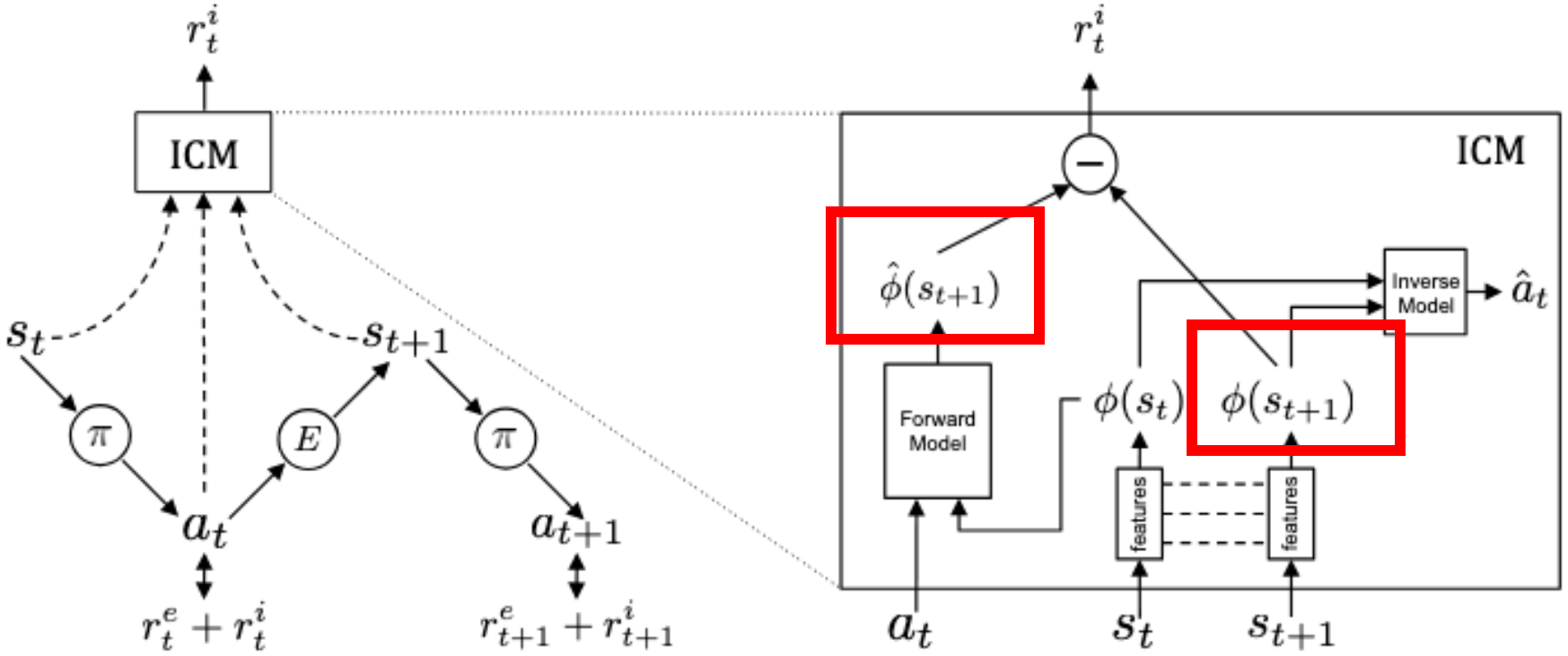
- $\phi(\cdot)$ is a feature encoder

Predict next state features



Predict $\phi(s_{t+1})$ given $\phi(s_t)$ and a_t

Use difference as a reward



Reward is obtained from $\hat{\phi}(s_{t+1}) - \phi(s_{t+1})$

Curiosity Driven Exploration by Self-Supervised Prediction

ICML 2017

Deepak Pathak, Pulkit Agrawal, Alexei Efros, Trevor Darrell
UC Berkeley

Extrinsic motivation (transfer learning)

Using a teacher to guide exploration

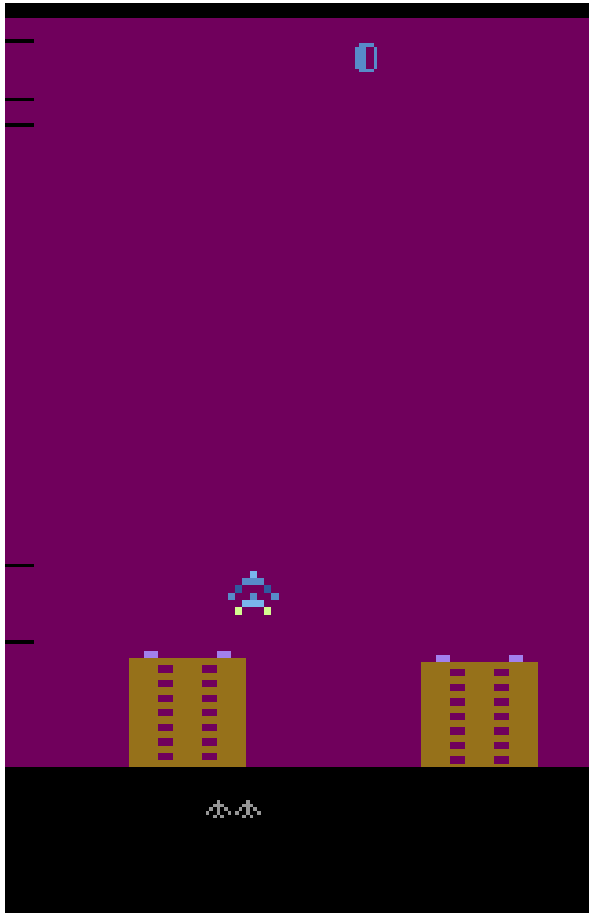
What if we have another agent that has learned a similar task?

- Task is *different* but shares some things in common

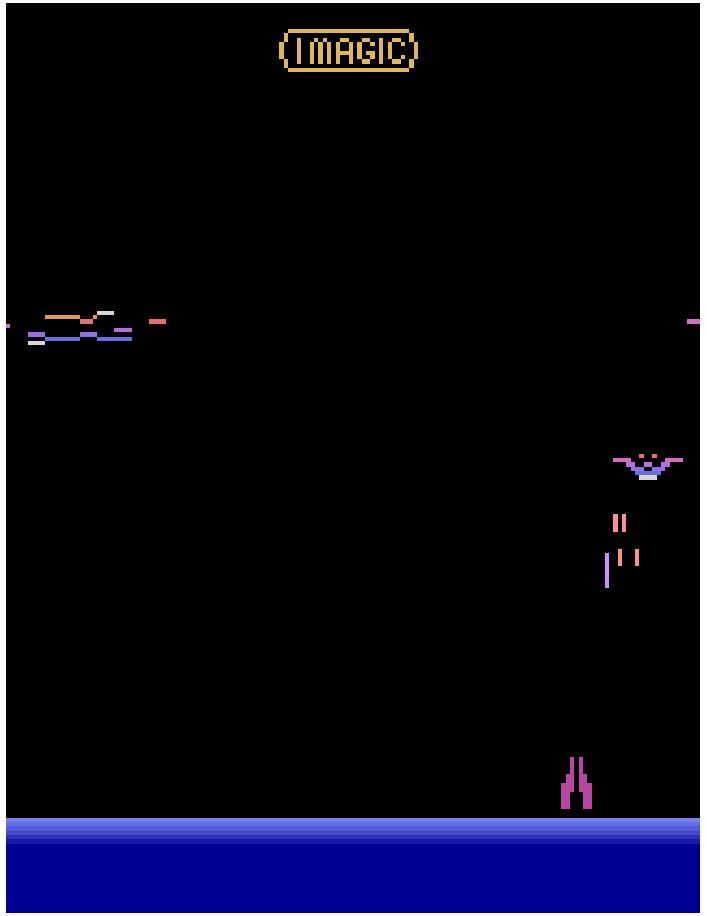
Can this agent (teacher) help our agent learn its own task?

Idea: teacher provides bonus reward to student when it enters states that are beneficial

Example in Atari



Air Raid
(Teacher)



Demon Attack
(Student)

Policy distillation

Distill the teacher's policy into the student's via bonus rewards

Use the teacher's critic to compute difference between current state and previous state

$$\text{Bonus} = V_{Teach}(s_{t+1}) - V_{Teach}(s_t)$$

Czarnecki et al, Distilling Policy Distillation, 2019.

Jump-start RL

Explore using the teacher's policy

- Slowly transition from teacher to student as student improves
- Early learning = mostly teacher
- Late learning = mostly student

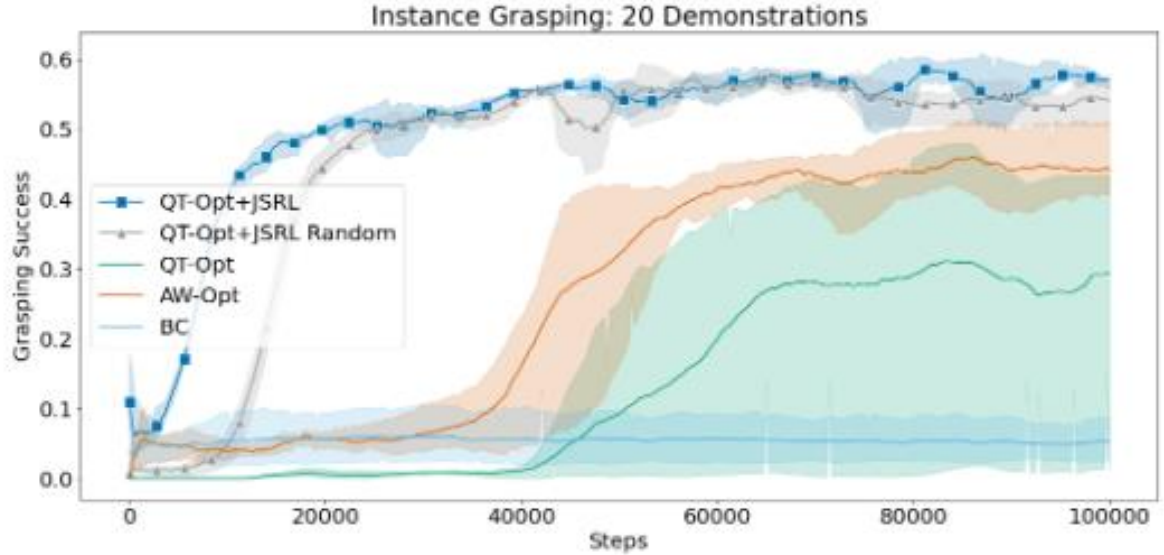
Unlike distillation, the teacher can be trained using imitation learning

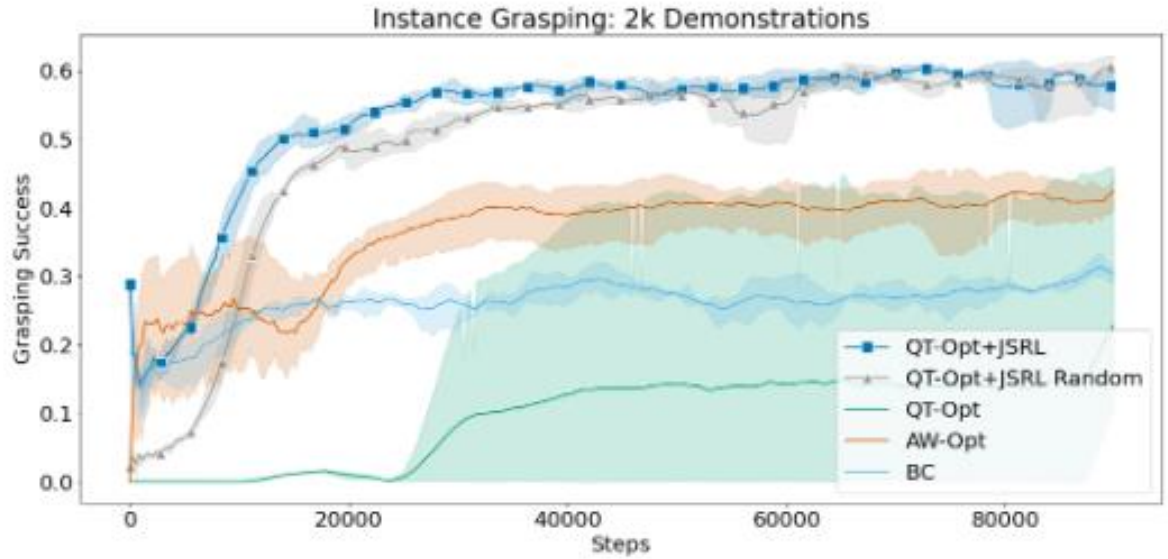
- Only needs access to an offline dataset

Uchendu et al, Jump-Start Reinforcement Learning, 2023.

Algorithm 1 Jump-Start Reinforcement Learning

Input: guide-policy π^g, performance threshold β, task horizon H, a sequence of initial guide-steps $H_1, H_2, \cdots, H_n$, where $H_i \in [H]$ for all $i \leq n$.
Initialize exploration-policy from scratch or with the guide-policy $\pi^e \leftarrow \pi^g$. Initialize Q-function $\hat{Q}$ and dataset $\mathcal{D} \leftarrow \varnothing$.
for current guide step $h = H_1, H_2, \cdots, H_n$ **do**
    Set the non-stationary policy $\pi_{1:h} = \pi^g$, $\pi_{h+1:H} = \pi^e$
    Roll out the policy π to get trajectory $\{(s_1, a_1, r_1), \cdots, (s_H, a_H, r_H)\}$; Append the trajectory to the dataset $\mathcal{D}$.
    $\pi^e, \hat{Q} \leftarrow \text{TRAINPOLICY}(\pi^e, \hat{Q}, \mathcal{D})$
    if $\text{EVALUATEPOLICY}(\pi) \geq \beta$ **then**
        Continue
    end if
end for





What are the problems with these methods?

What are the problems with these methods?

They assume that the student's task is sufficiently in-distribution for the teacher to act upon!

What happens if this isn't the case and the teacher gives bad advice?

- Can completely cripple the student's exploration
- Repeatedly lead the student to low reward regions
- May get stuck in local minima and unable to break out even after teacher stops giving advice

The following slides are taken from my CoLLAs 2023 presentation

Introspective transfer learning

What if the teacher introspects over its own policy before giving advice?

- If advice doesn't help the student, don't give it!

Goal: avoid issuing bad advice

How do we know if advice helps?

- Compute expected reward in student's environment
- Out-of-distribution detection

Campbell et al, Introspective Action Advising for Interpretable Transfer Learning, 2023

Contribution: Introspective Action Advising

When is advice
beneficial?

Definition: Given a student's action-value Q_{π^S}, advice is beneficial when...

$$Q_{\pi^S}(s, a^T) > Q_{\pi^S}(s, a^S)$$

Teacher's action

Student's action

Key Ideas

Advice is **transferable** if the expected return is ”close enough” in the source and target tasks

$$|Q_{\pi^T}^{\text{Tar}}(s_t, a^T) - Q_{\pi^T}^{\text{Src}}(s_t, a^T)| \leq \epsilon$$

Target task — Source task — Threshold

$$\Downarrow$$

$$|V_{\pi^T}^{\text{Tar}}(s_t) - V_{\pi^T}^{\text{Src}}(s_t)| \leq \epsilon$$

Transferable advice is **beneficial** if issued sufficiently early in training

Estimating Target Value

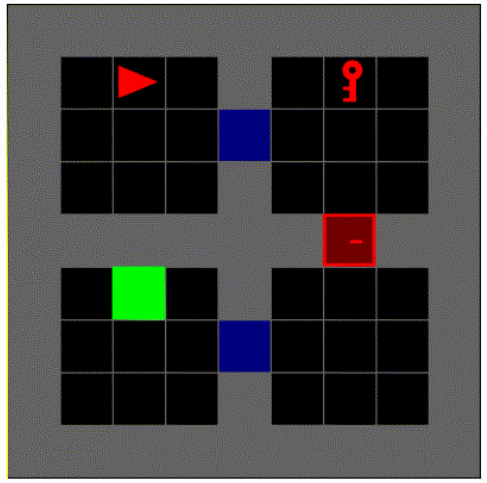
$$|V_{\pi^T}^{\mathrm{Tar}}(s_t) - V_{\pi^T}^{\mathrm{Src}}(s_t)| \leq \epsilon$$

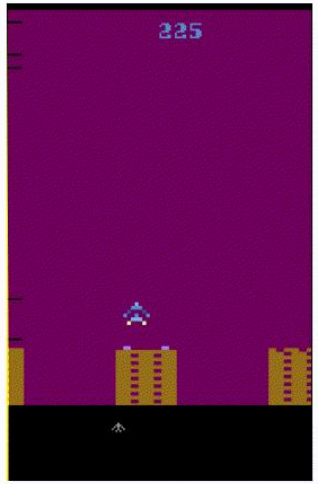
Fine-tune a copy of the teacher's state-value function on student observed returns

Assumption: sufficiently **out-of-distribution** states will yield large variance in $V_{\pi^T}^{\mathrm{Src}}$ estimates and exceed ϵ

Experiments

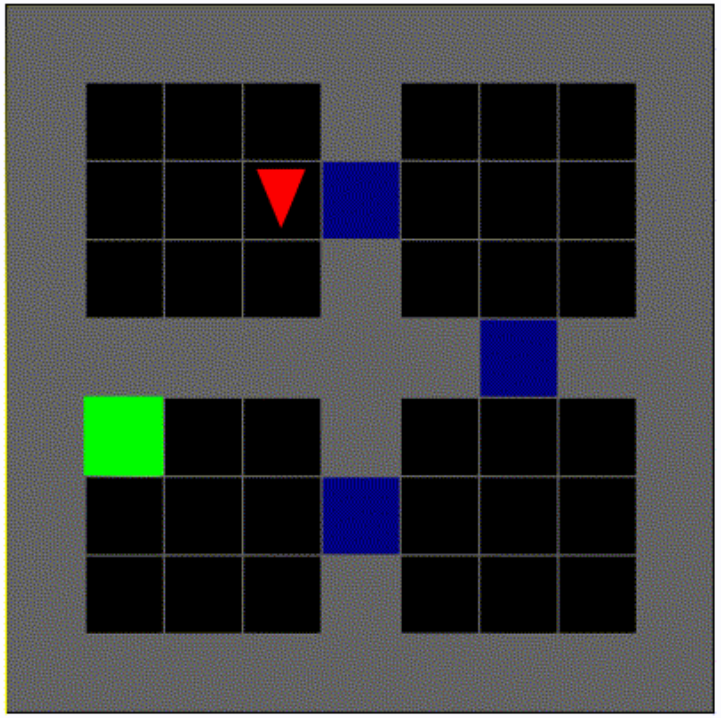
1) Does IAA yield positive transfer between tasks?
2) How does IAA compare to prior methods?
3) Is transferred knowledge interpretable?



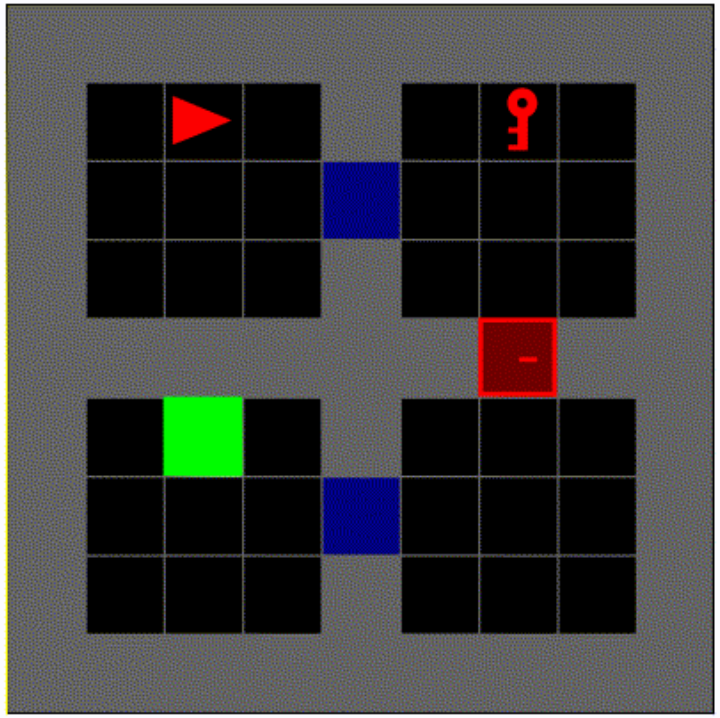


GridWorld – Goal Navigation

Key + Door



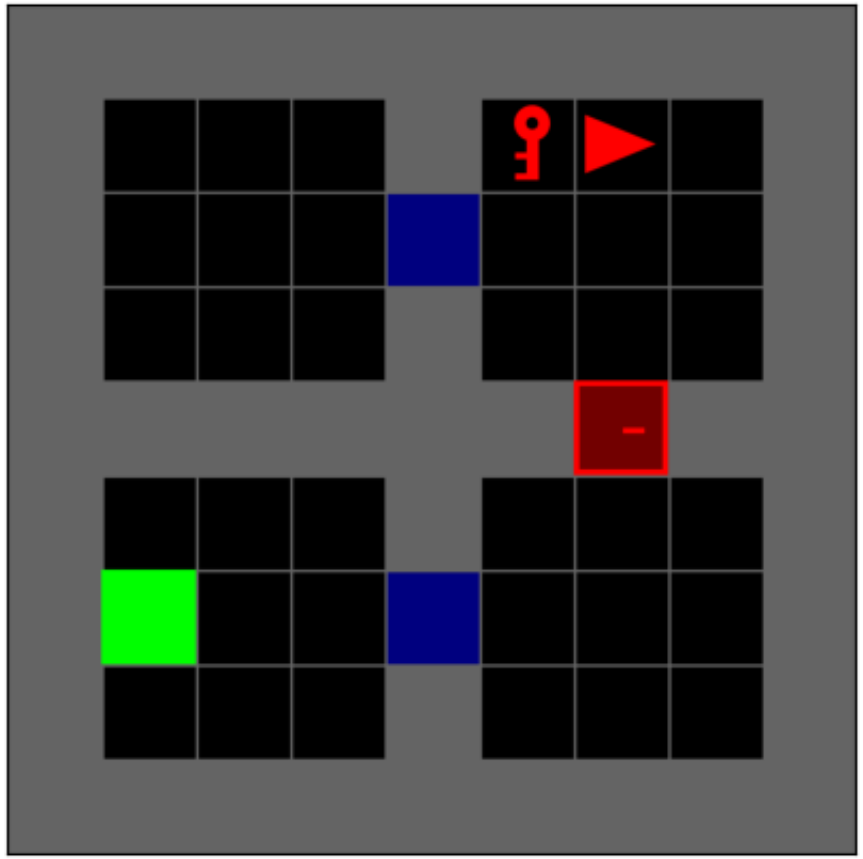
Source



Target

Training trajectory

$t = 0$



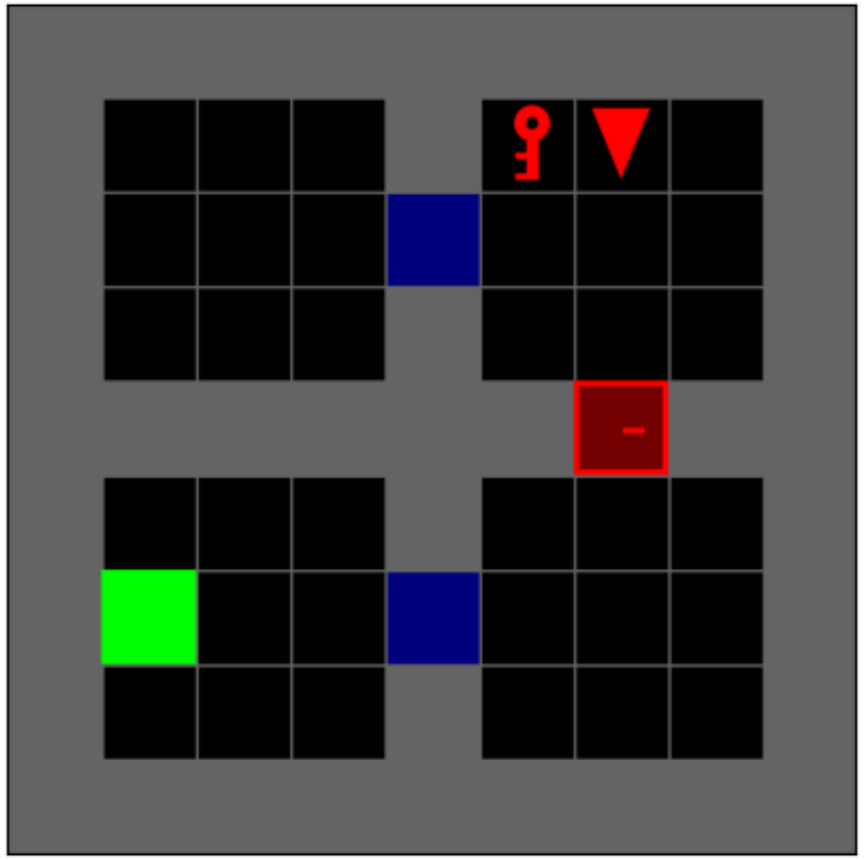
Teacher: Forward

Student: Turn right

Teacher's advice is not **transferable** before locked door

Training trajectory

***t* = 1**

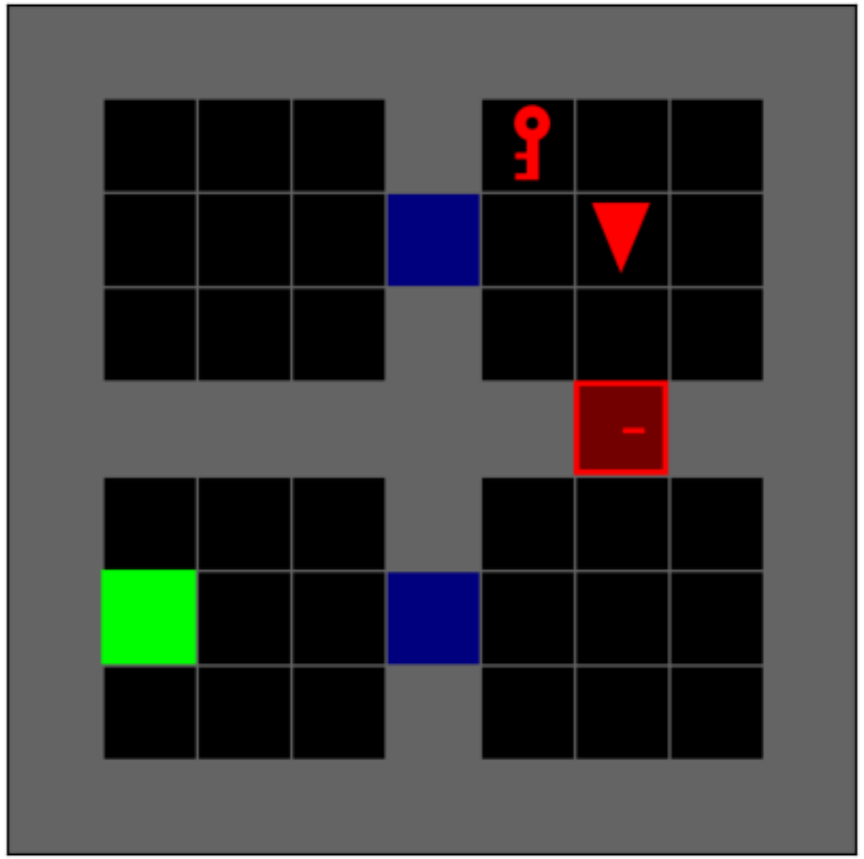


Teacher: Forward

Student: Forward

Training trajectory

$t = 2$

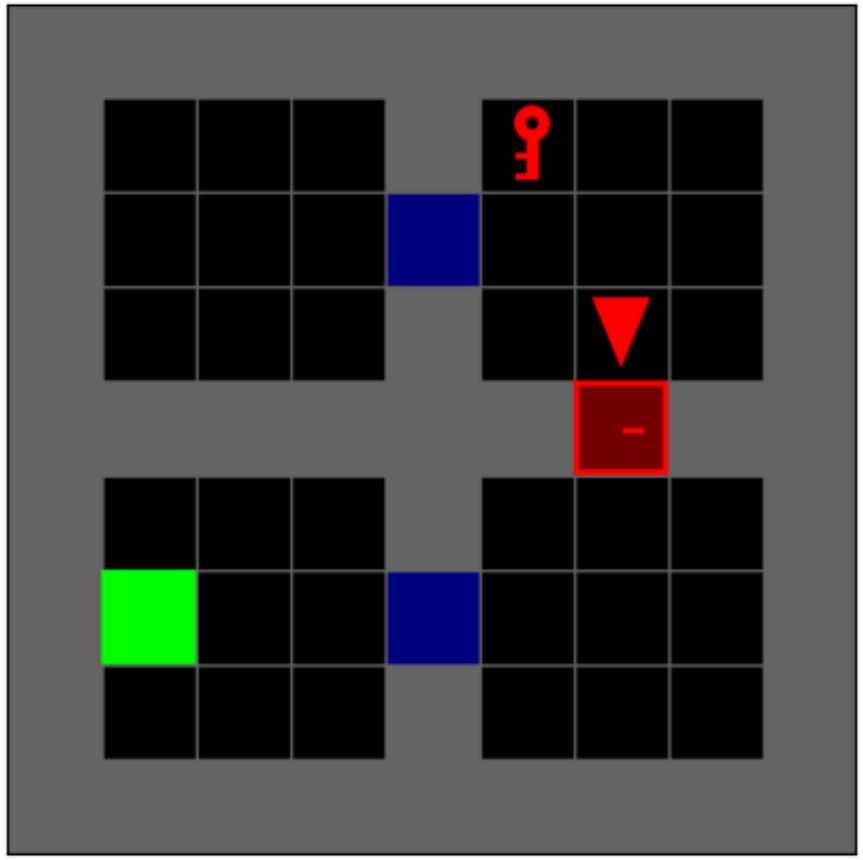


Teacher: Forward

Student: Forward

Training trajectory

$t = 3$



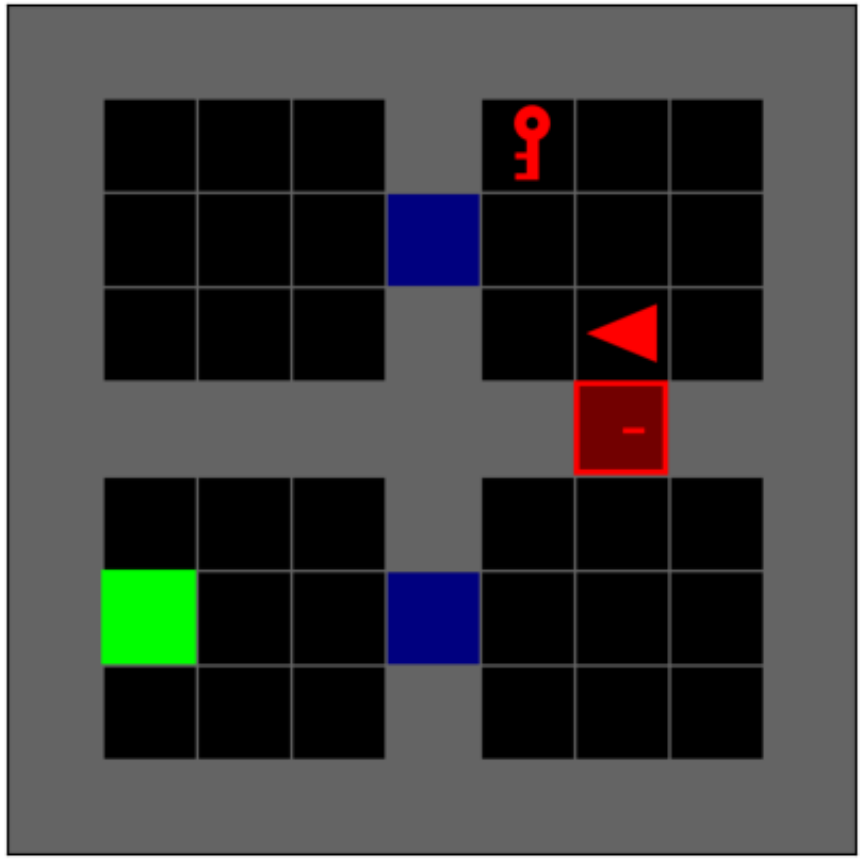
Teacher: Forward

Student: Turn right

Teacher wants to move forward as it doesn't know about doors

Training trajectory

$t = 4$

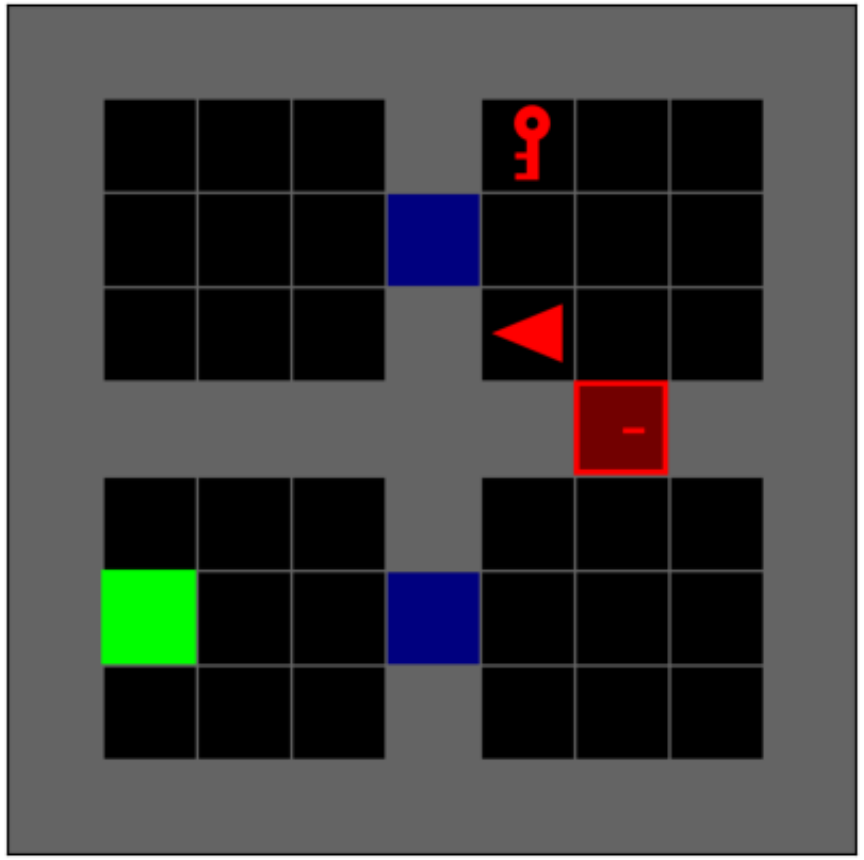


Teacher: Forward

Student: Forward

Training trajectory

$t = 5$

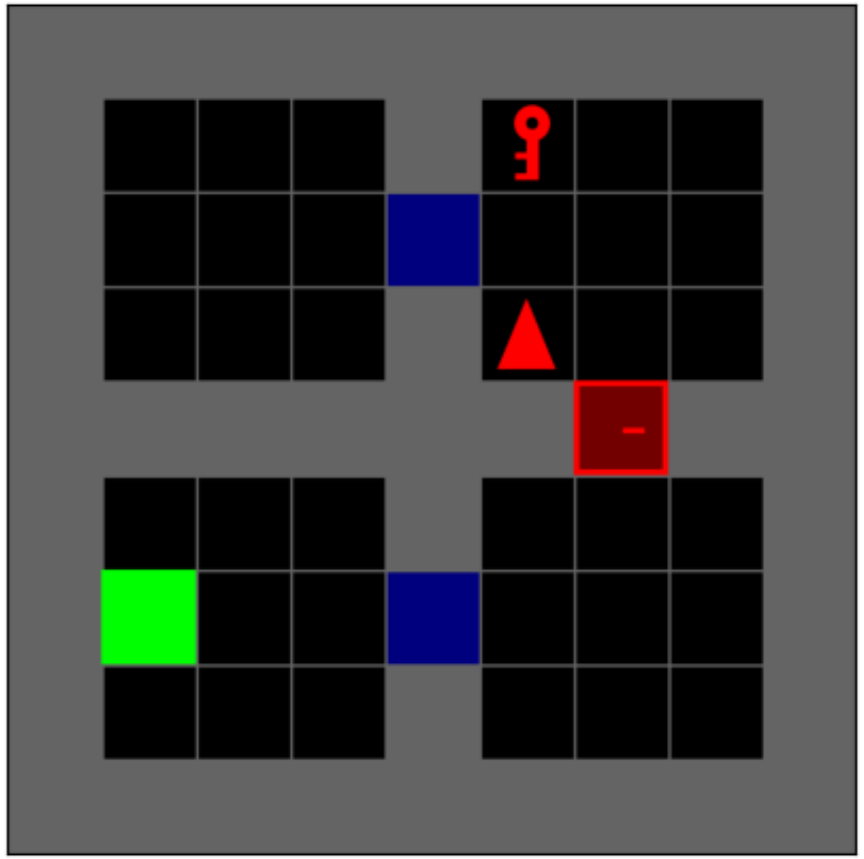


Teacher: Turn left

Student: Turn right

Training trajectory

$t = 6$

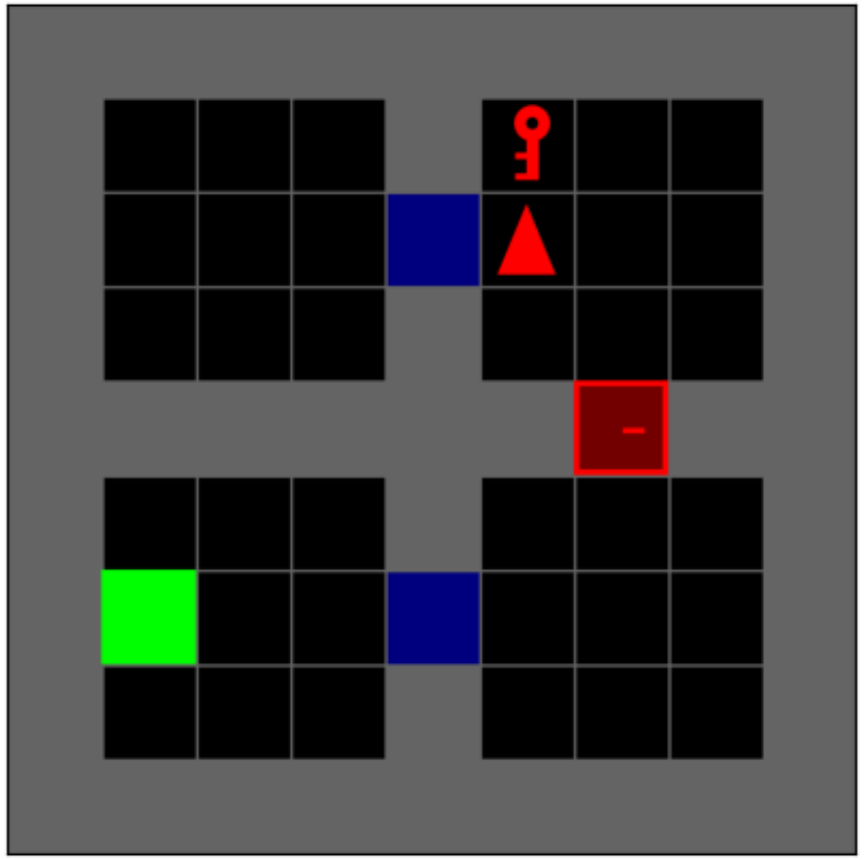


Teacher: Turn left

Student: Forward

Training trajectory

***t* = 7**

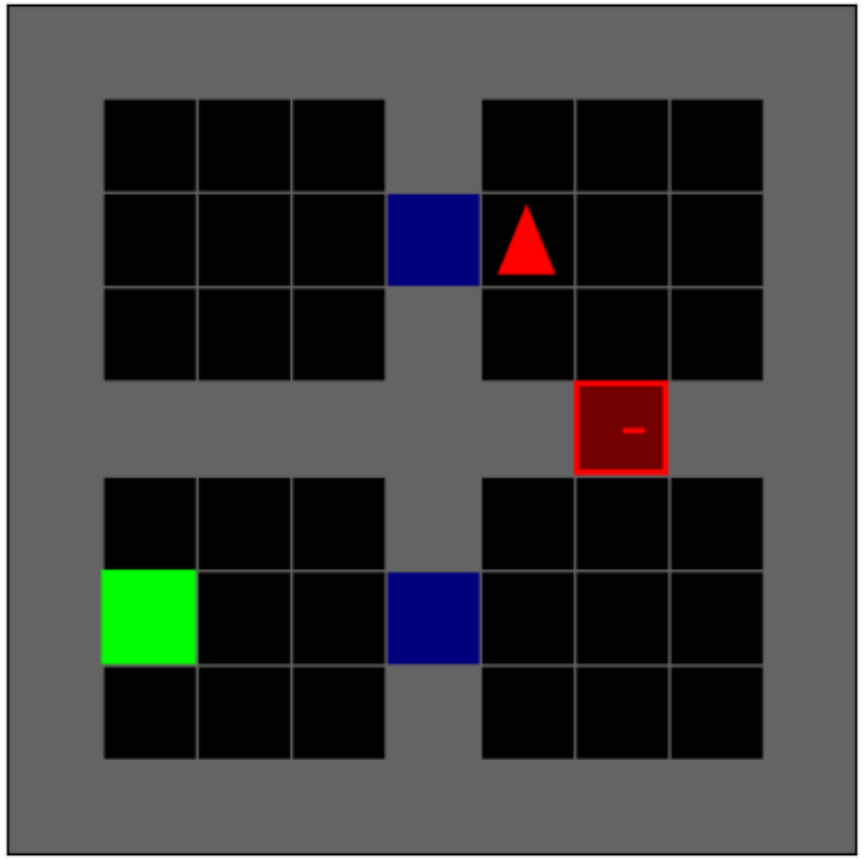


Teacher: Turn left

Student: Pick up

Training trajectory

$t = 8$

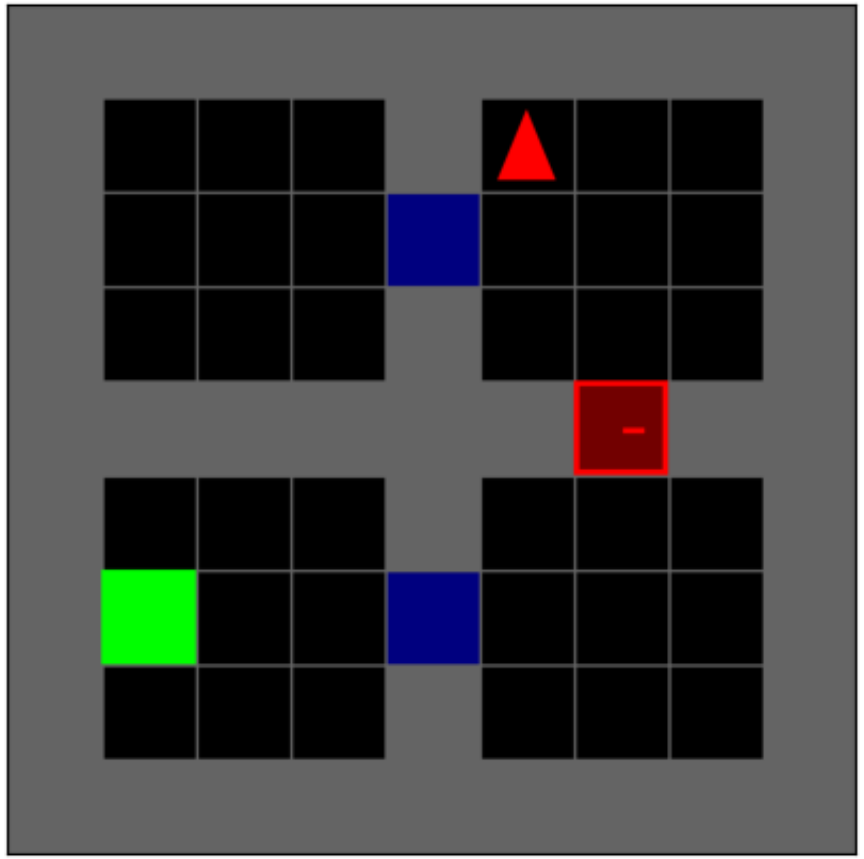


Teacher: Turn left

Student: Forward

Training trajectory

$t = 9$

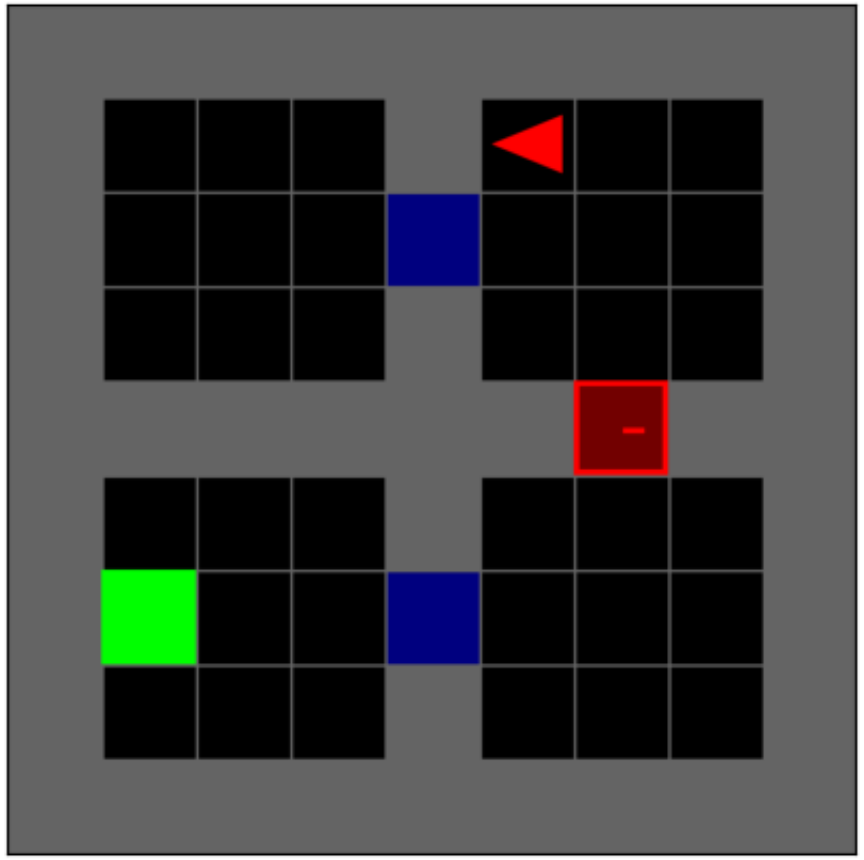


Teacher: Turn left

Student: Turn left

Training trajectory

$t = 10$

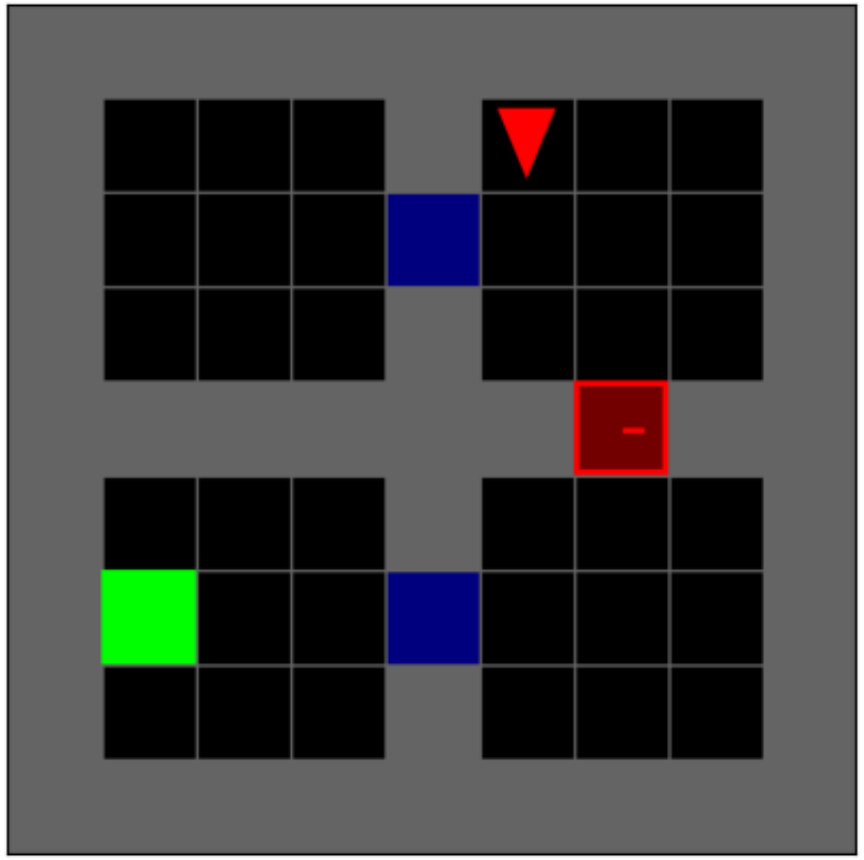


Teacher: Turn left

Student: Turn left

Training trajectory

$t = 11$

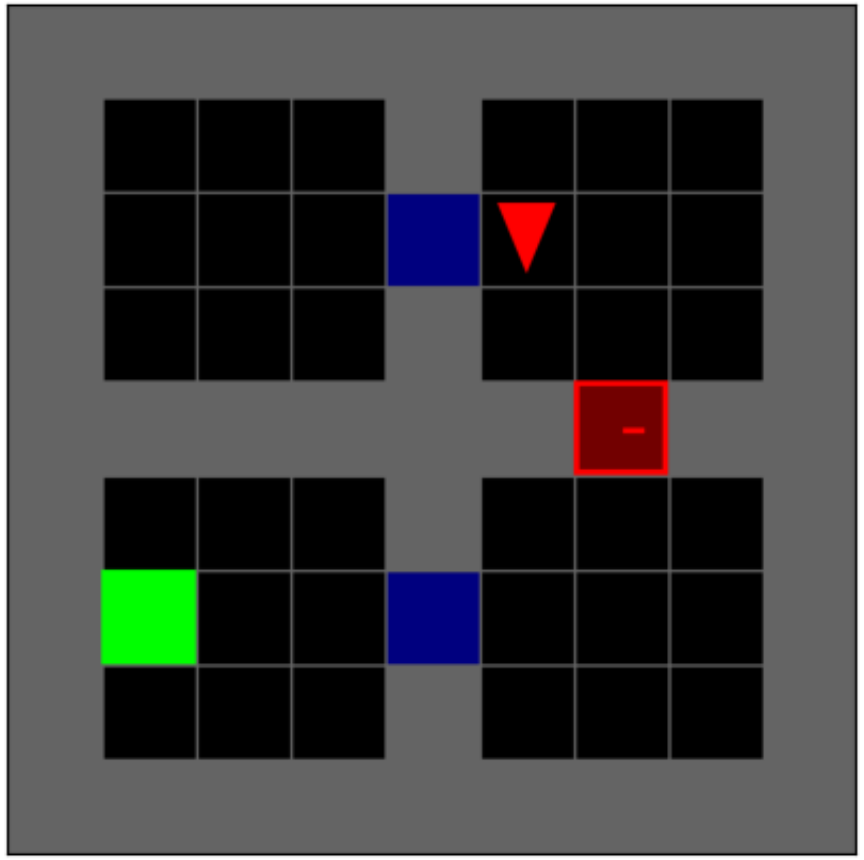


Teacher: Forward

Student: Forward

Training trajectory

$t = 12$

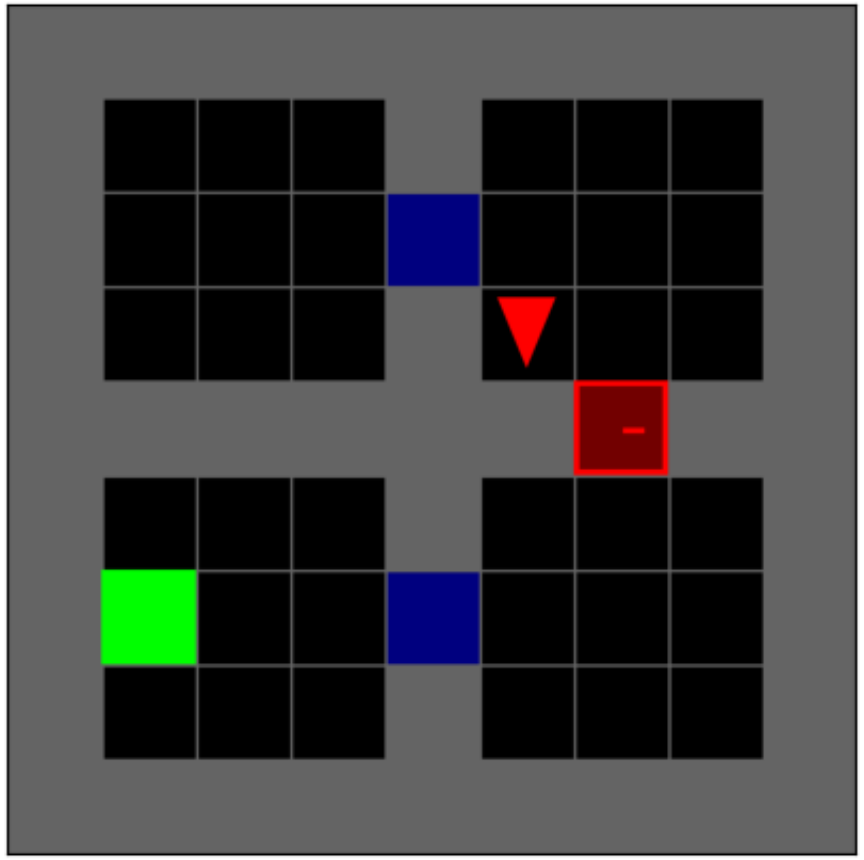


Teacher: Forward

Student: Forward

Training trajectory

$t = 13$

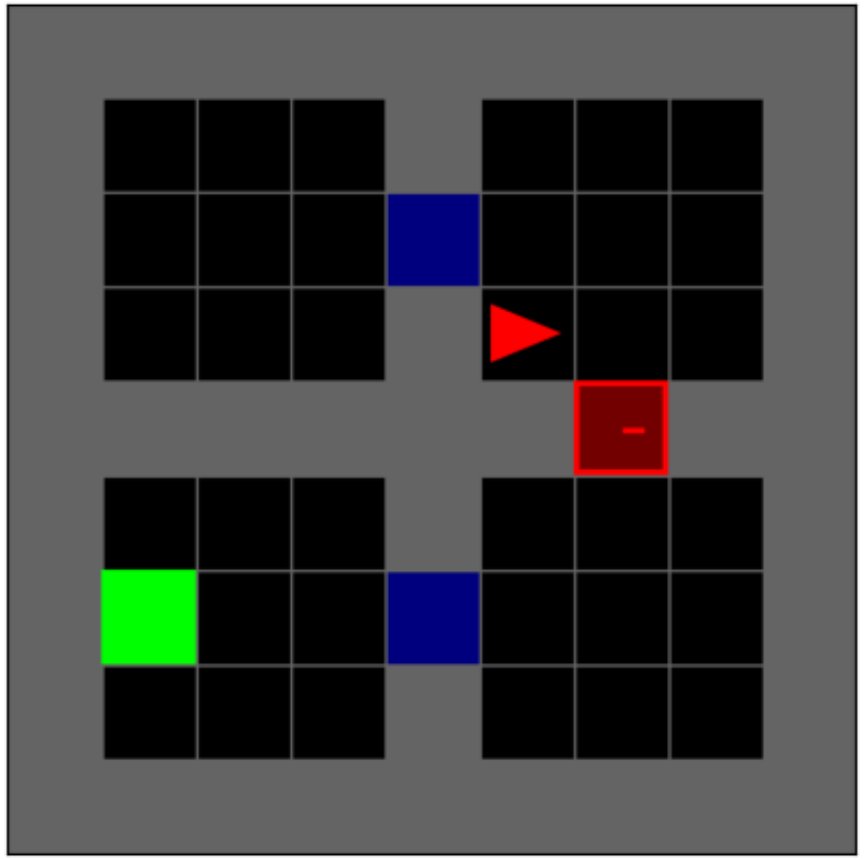


Teacher: Turn left

Student: Turn left

Training trajectory

$t = 14$

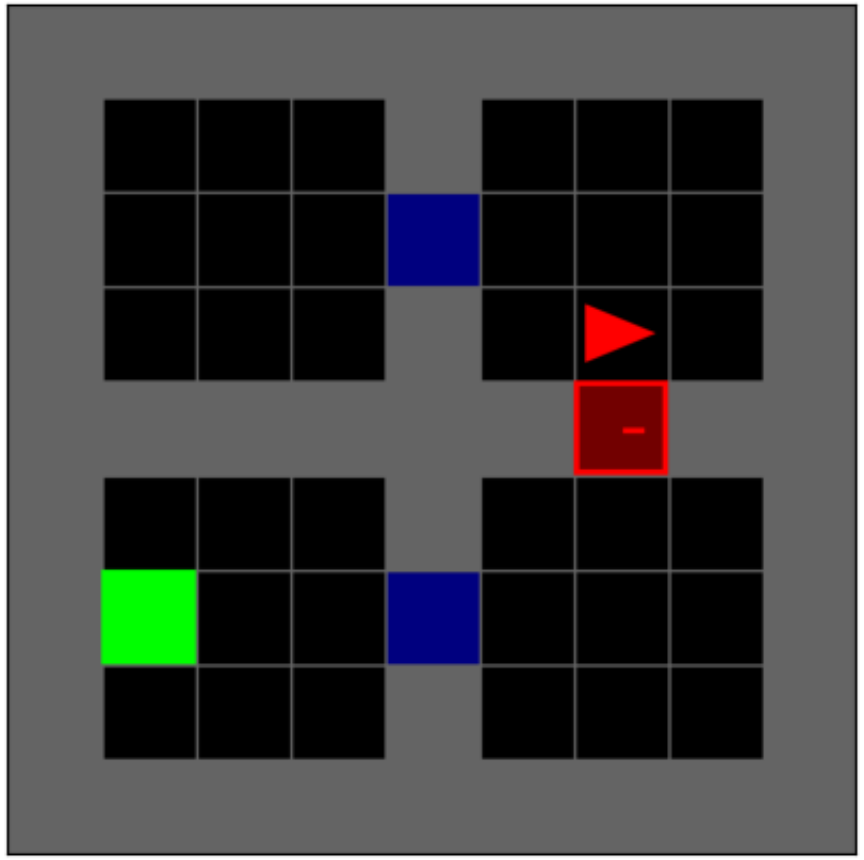


Teacher: Forward

Student: Forward

Training trajectory

$t = 15$

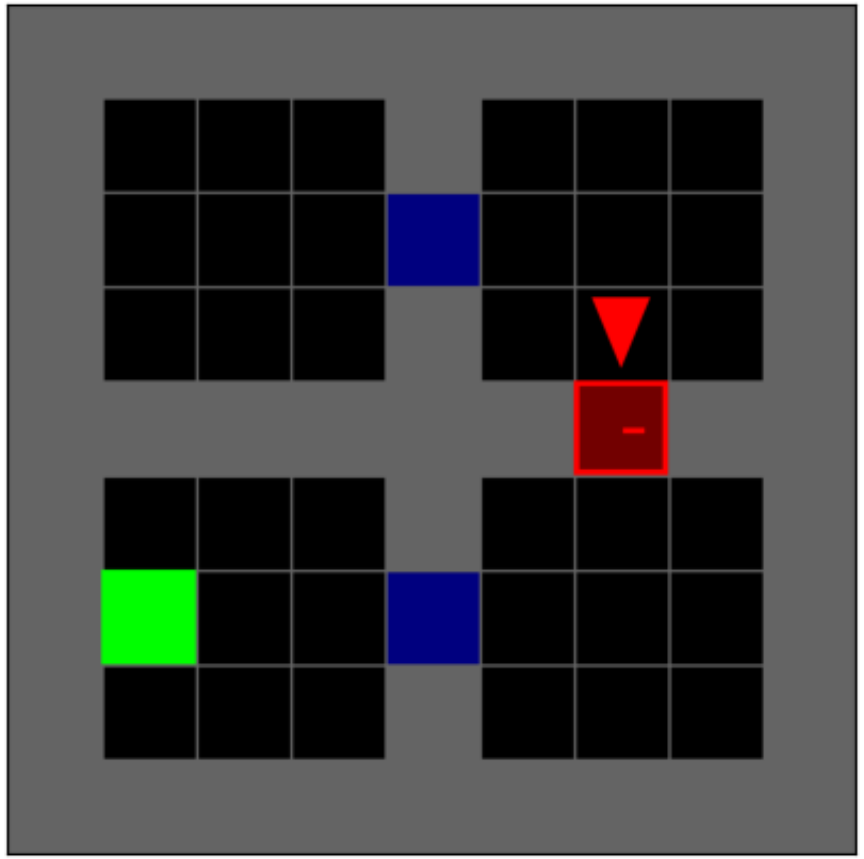


Teacher: Forward

Student: Turn right

Training trajectory

$t = 16$

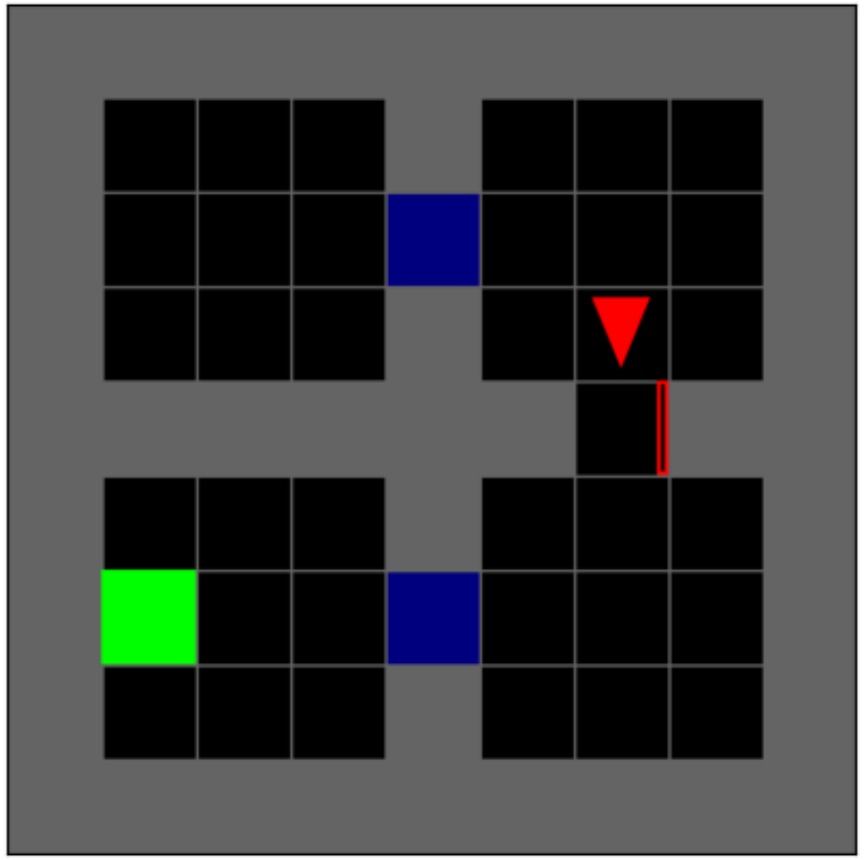


Teacher: Forward

Student: Toggle

Training trajectory

$t = 17$

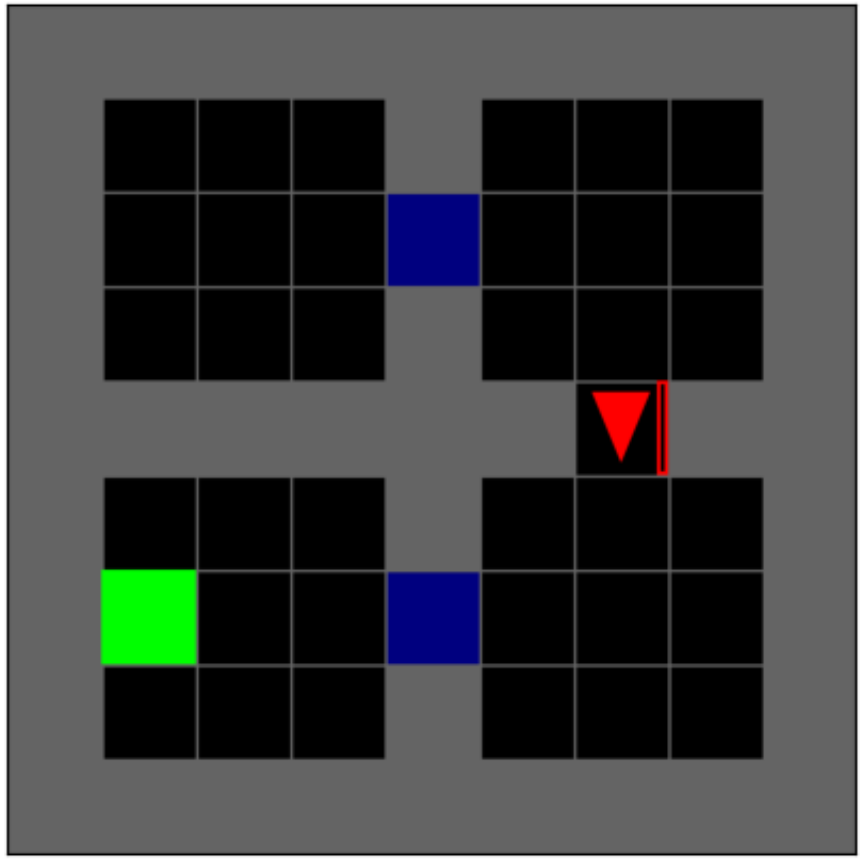


Teacher: Forward

Student: Forward

Training trajectory

***t* = 18**



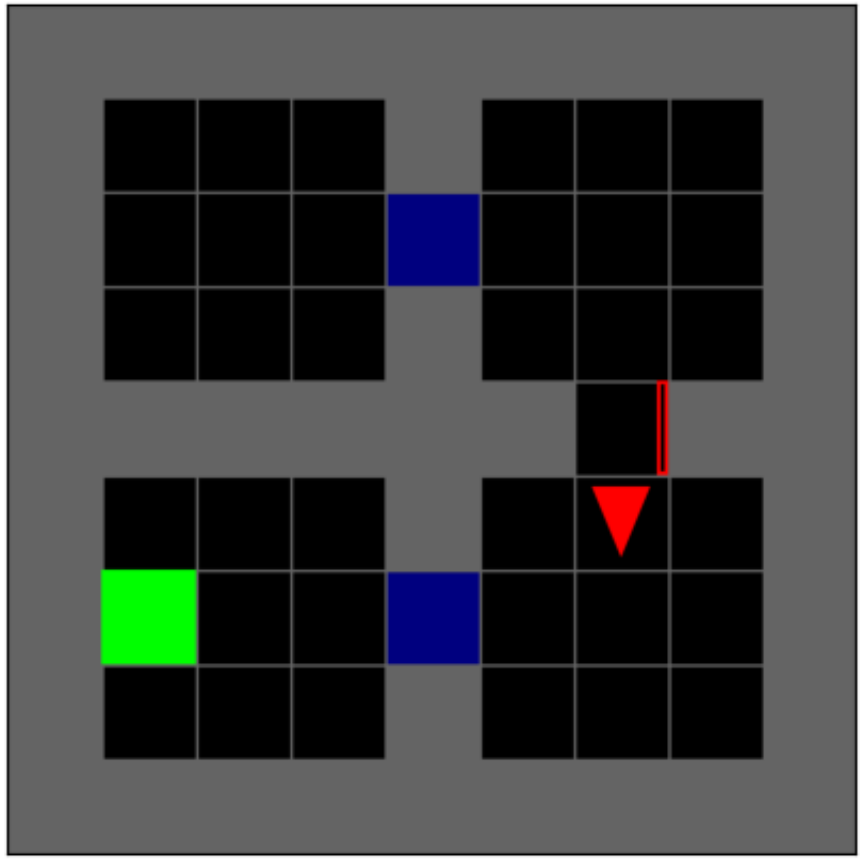
Teacher: Forward

Student: Forward

The teacher's advice is now **transferable**

Training trajectory

$t = 19$

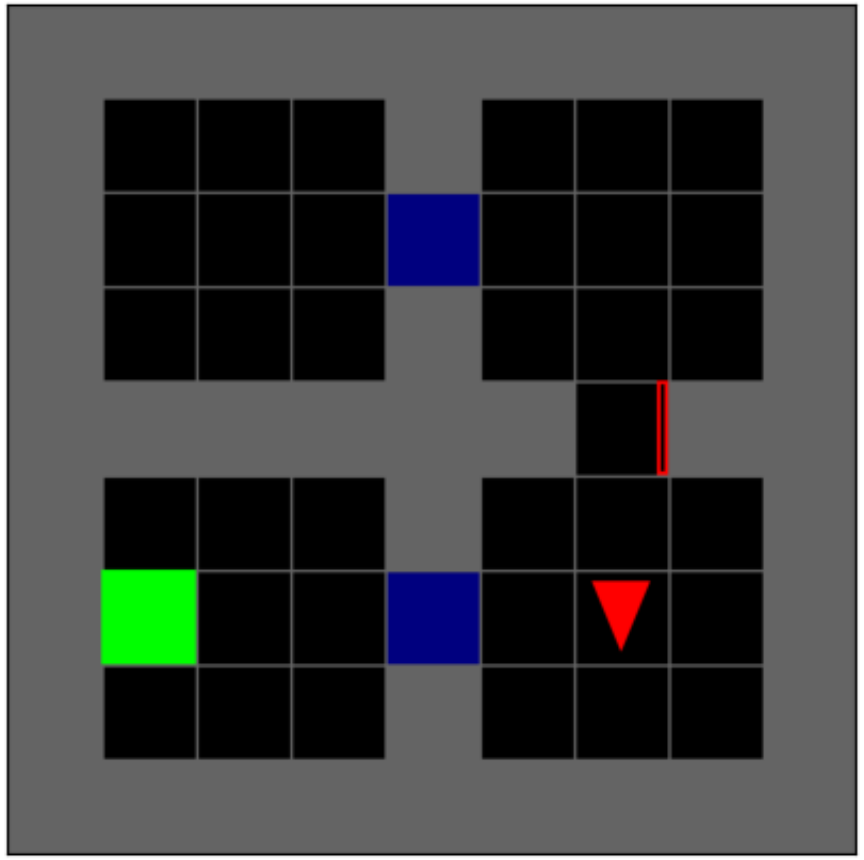


Teacher: Forward

Student: Forward

Training trajectory

***t* = 20**



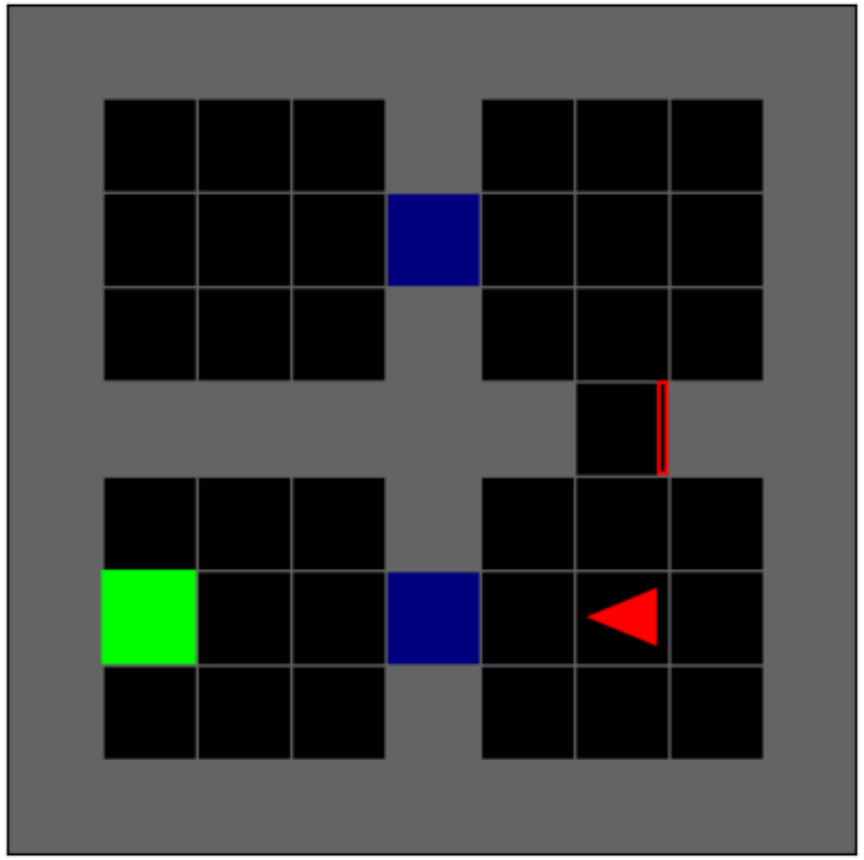
Teacher: Turn right

Student: Forward

Teacher's advice is **beneficial** and guides student correctly

Training trajectory

$t = 21$

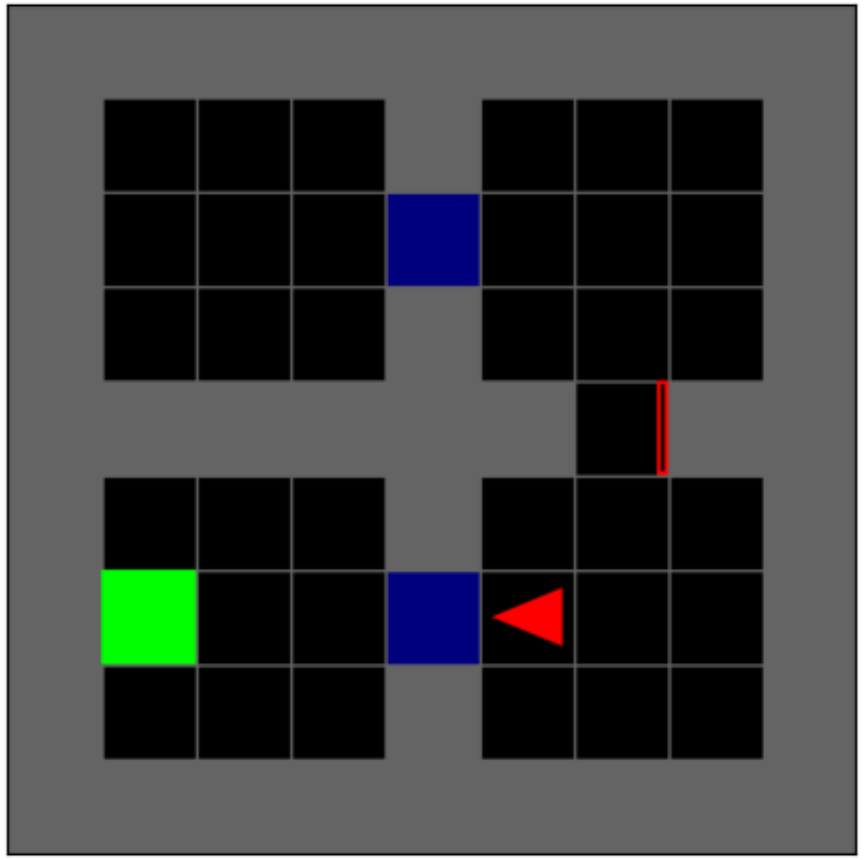


Teacher: Forward

Student: Forward

Training trajectory

$t = 22$

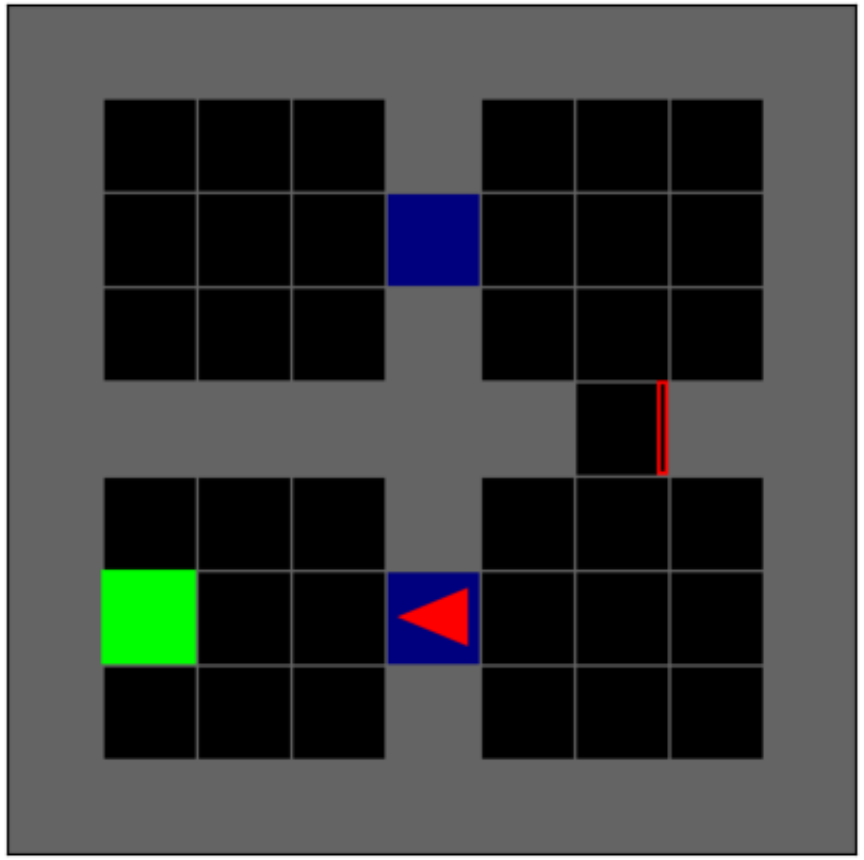


Teacher: Forward

Student: Forward

Training trajectory

$t = 23$

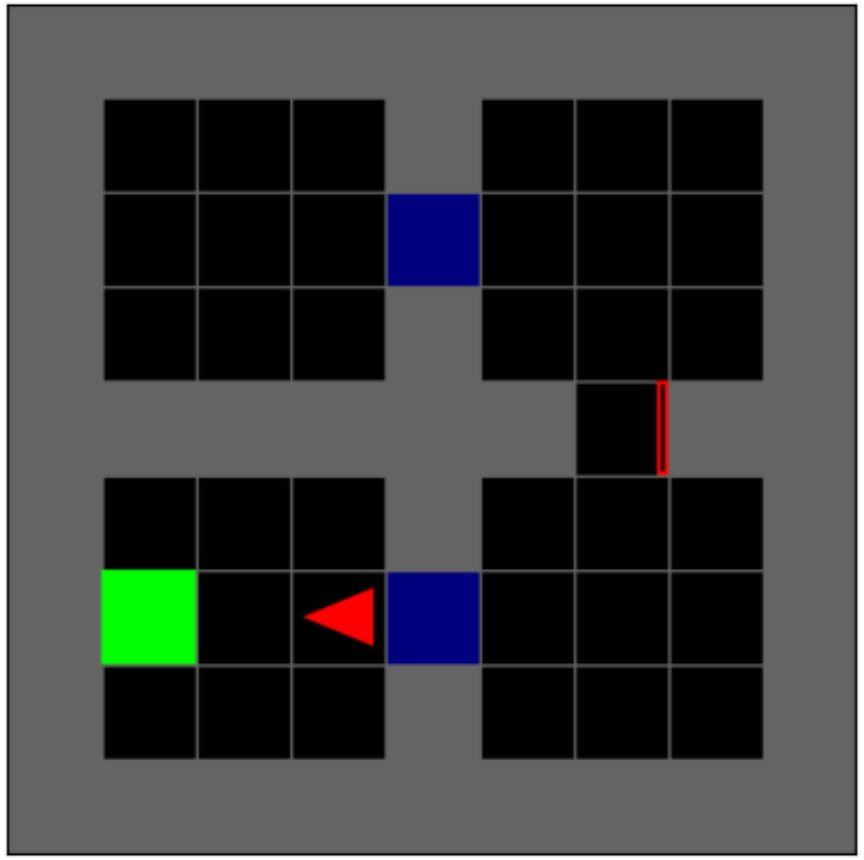


Teacher: Forward

Student: Forward

Training trajectory

$t = 24$

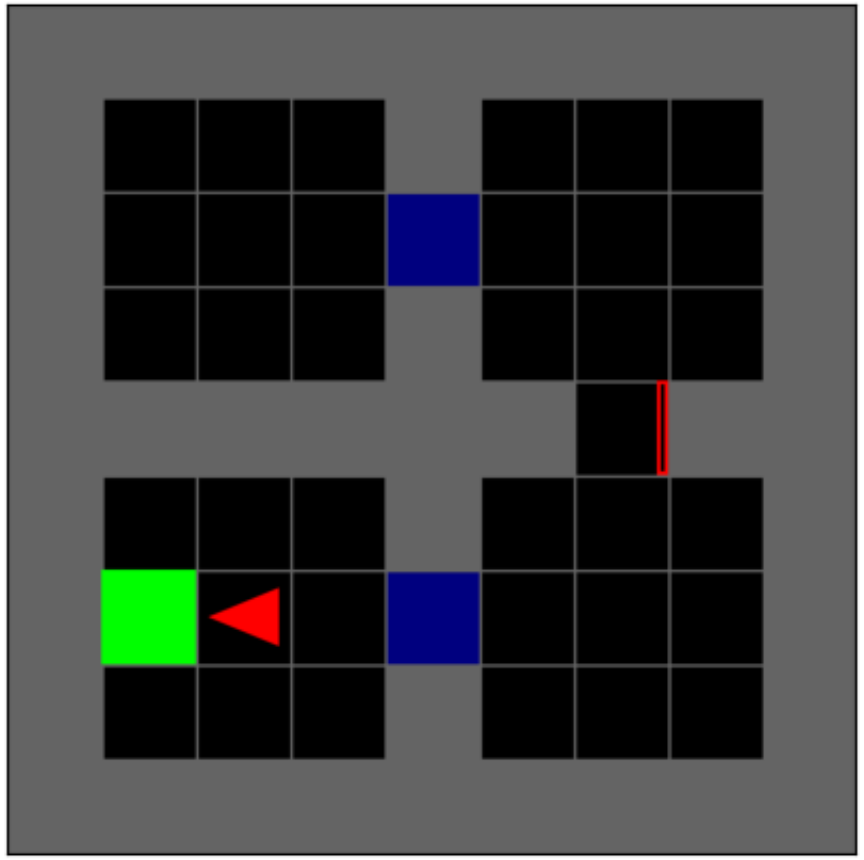


Teacher: Forward

Student: Forward

Training trajectory

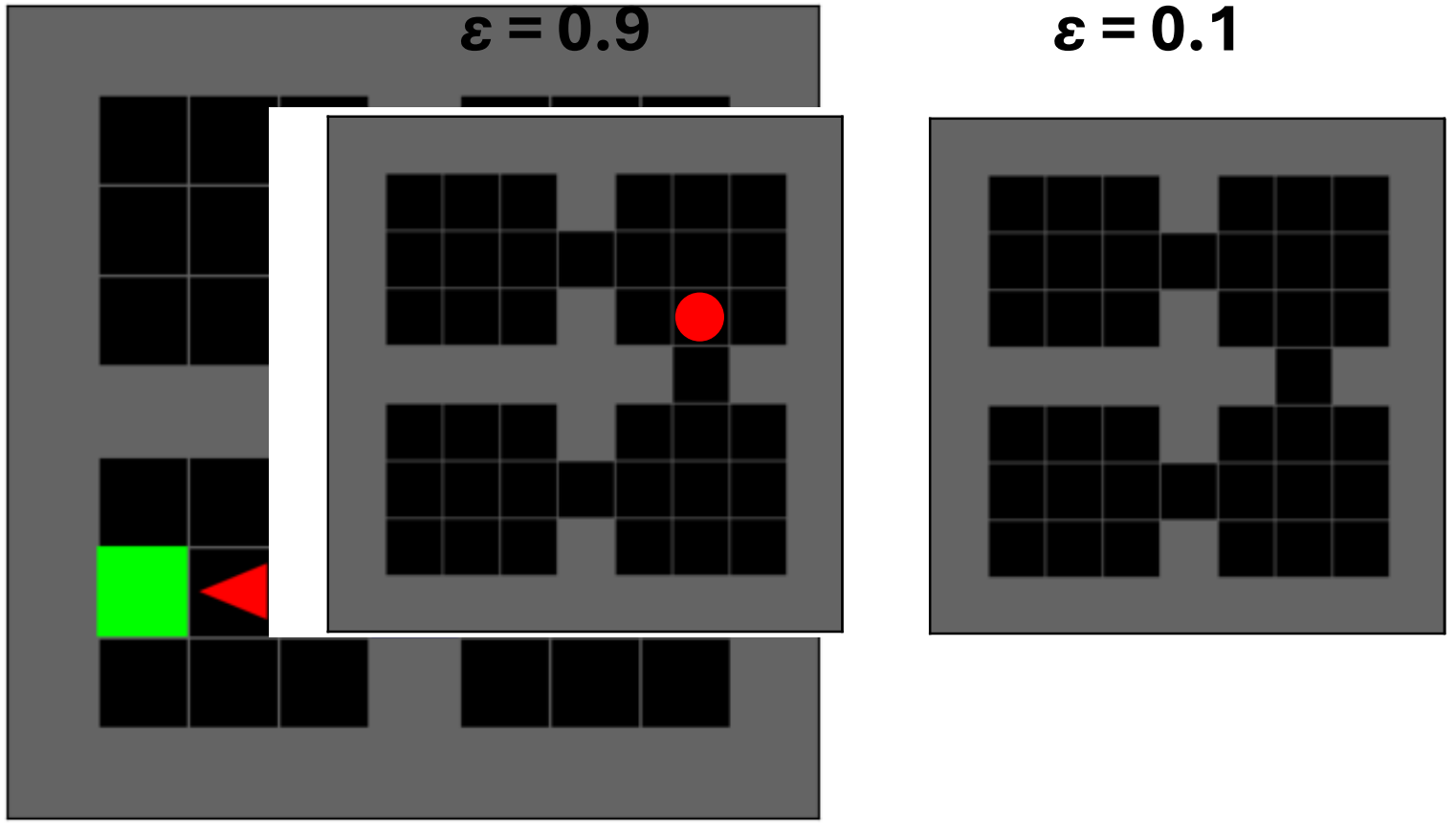
$t = 25$



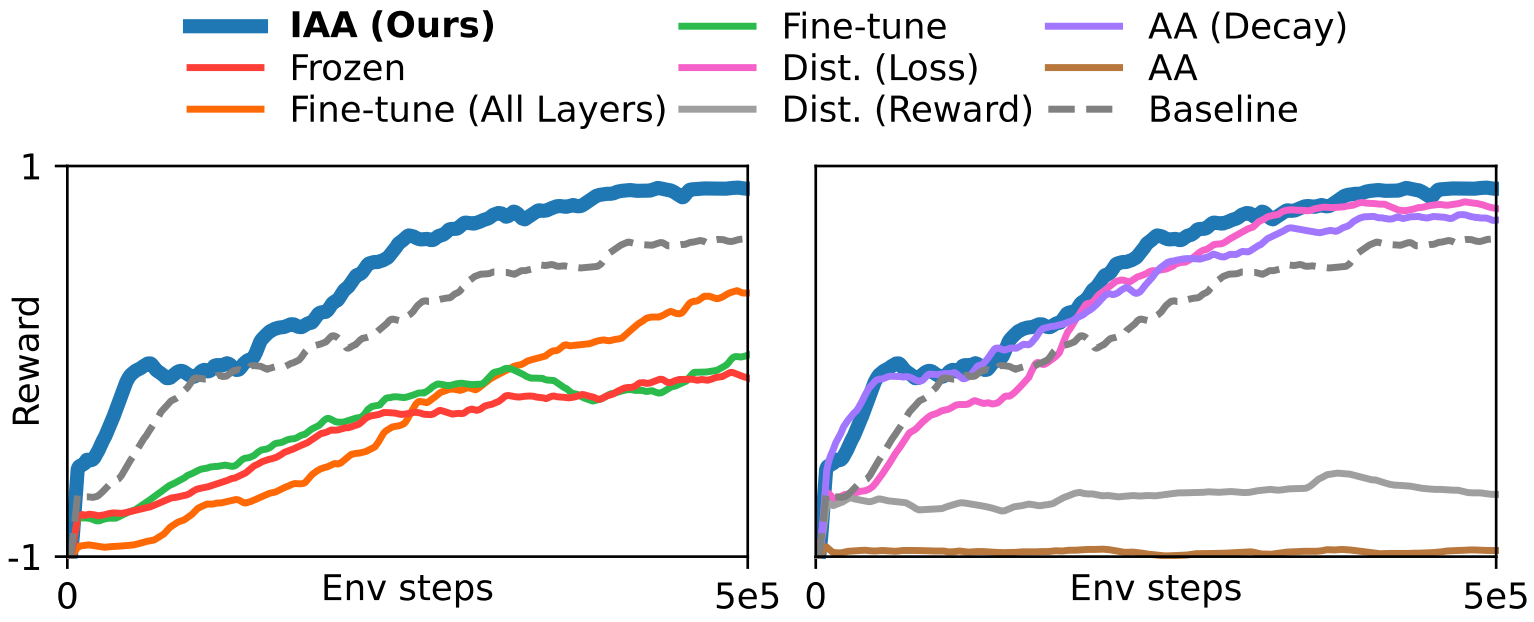
Teacher: Forward

Student: Forward

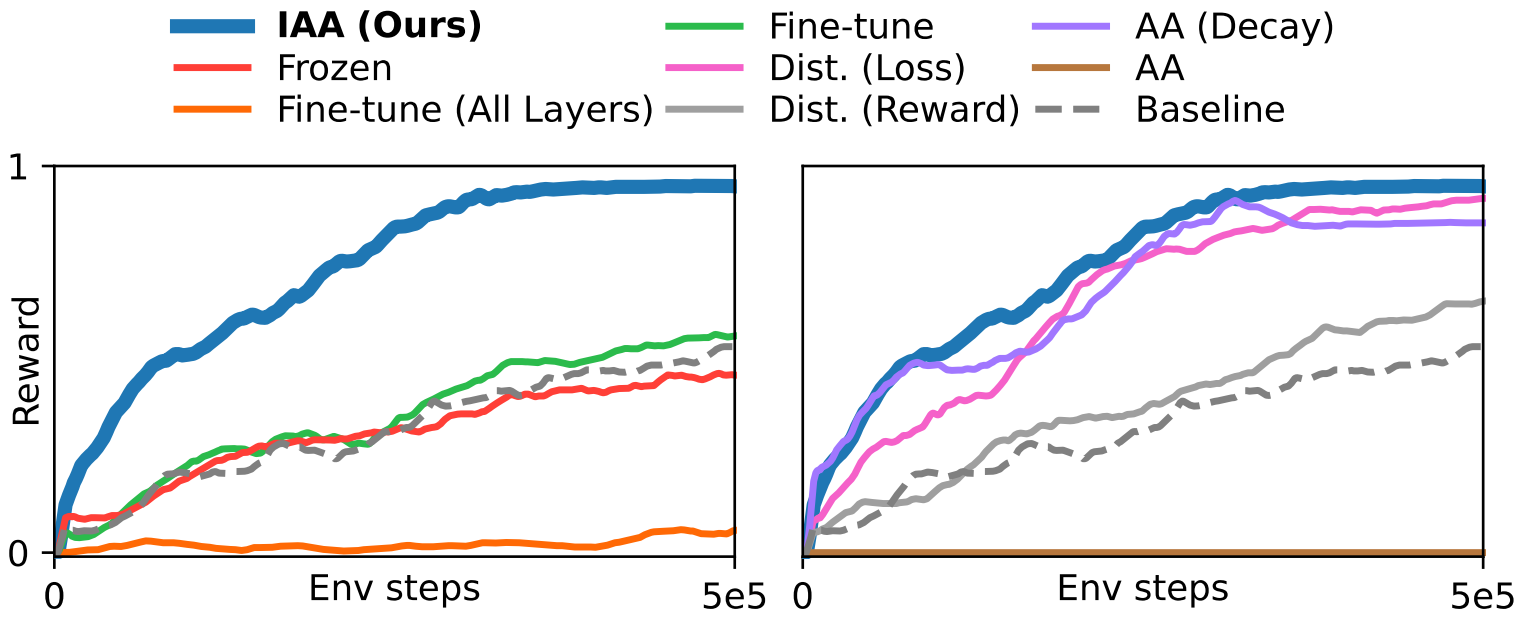
Training trajectory



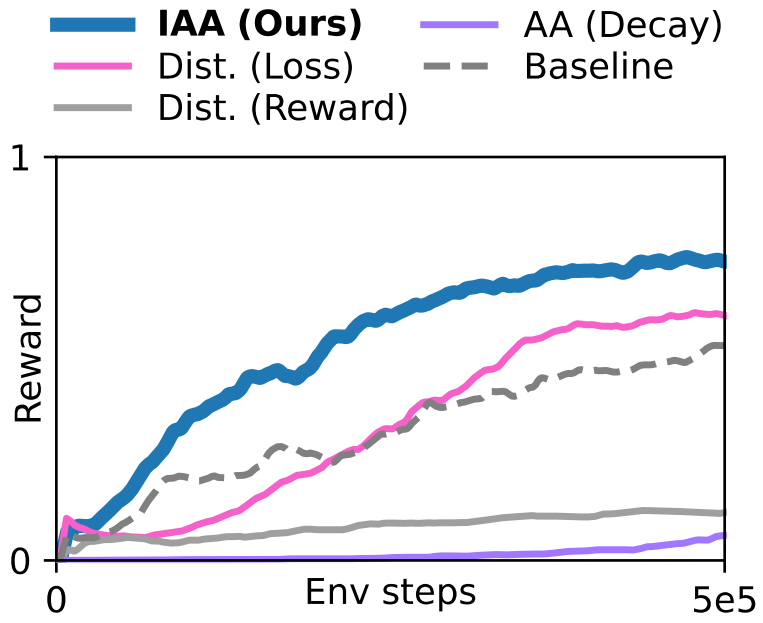
Dense Reward



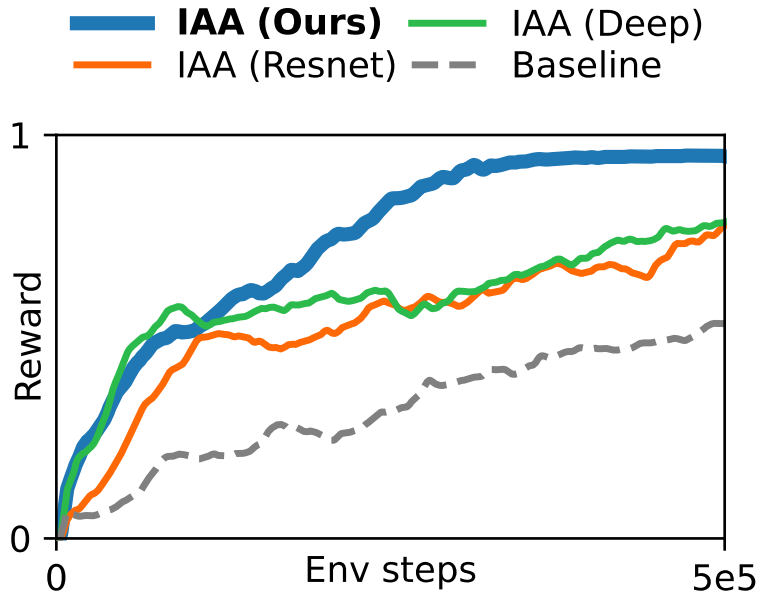
Sparse Reward



Unhelpful Teacher and Differing Architecture

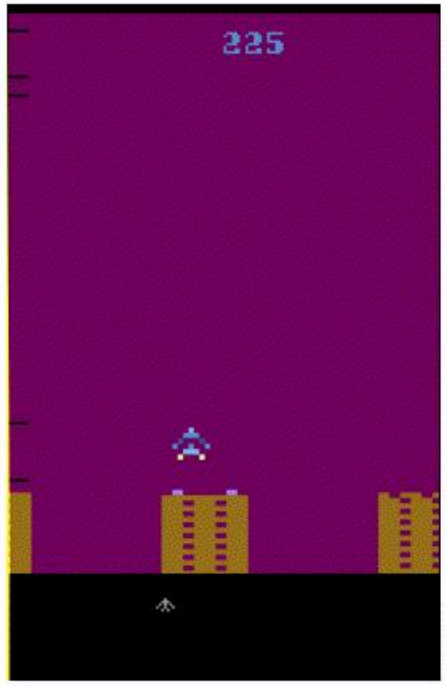


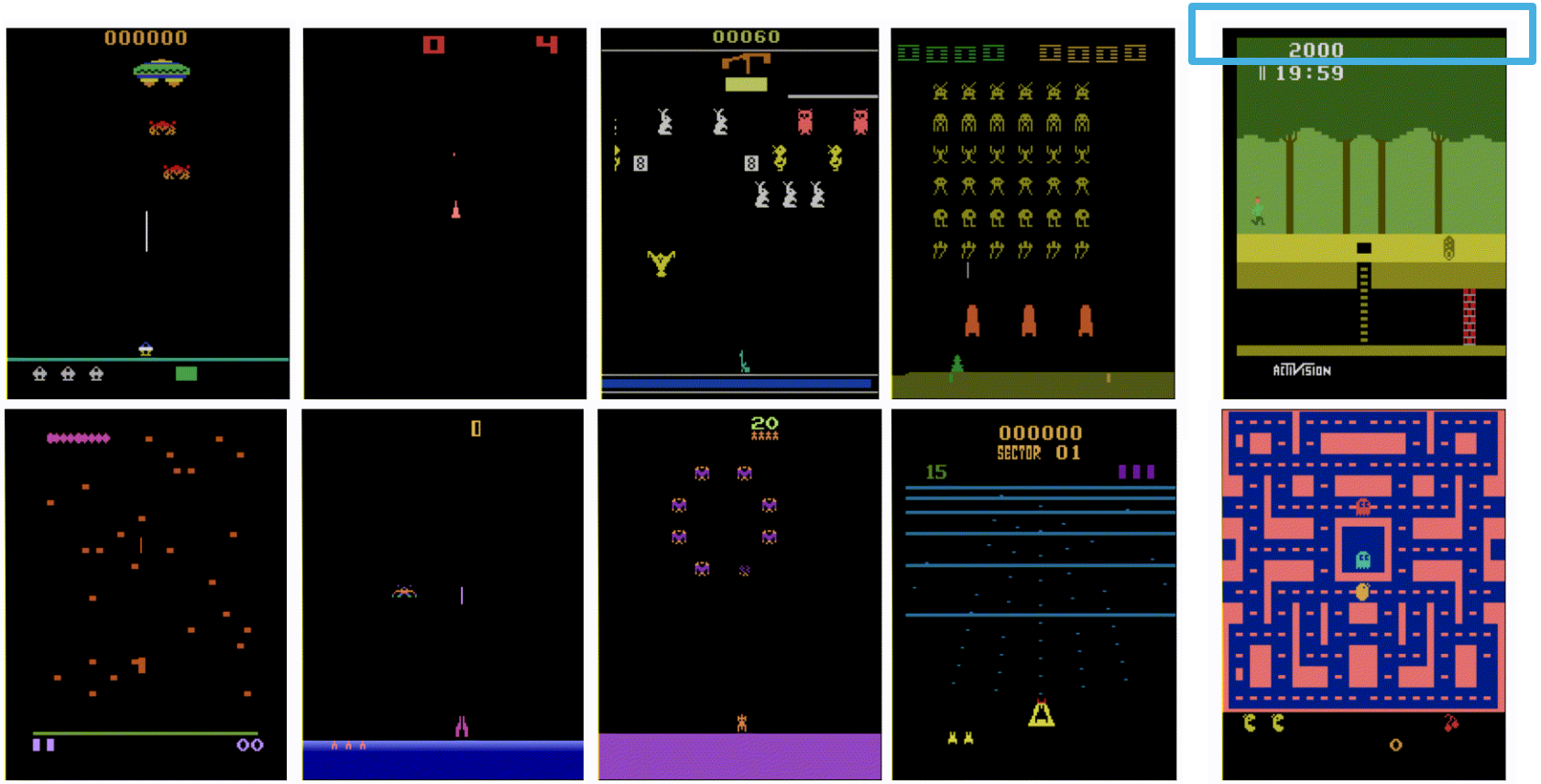
Unhelpful teacher



Differing architecture

Atari – Multiple Tasks



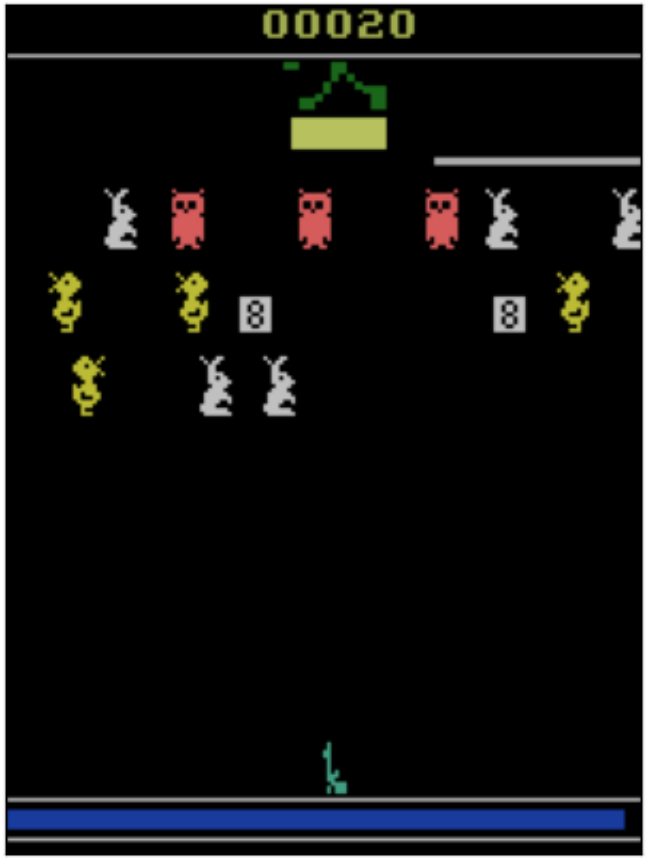


Source

Target

Training trajectory

$t = 0$



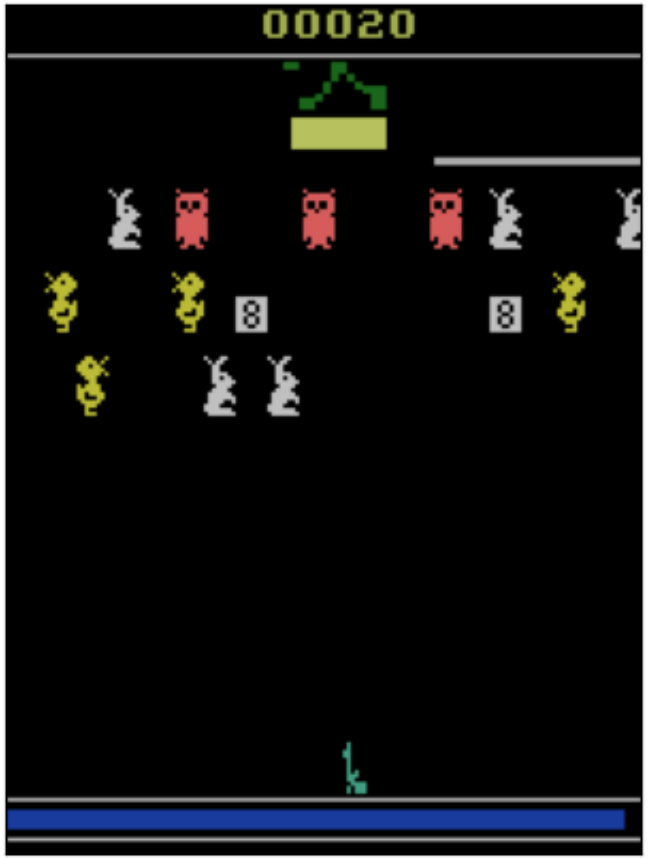
Teacher: Fire

Student: Move

Teacher's advice is only **transferable** for high-value states…

Training trajectory

$t = 1$

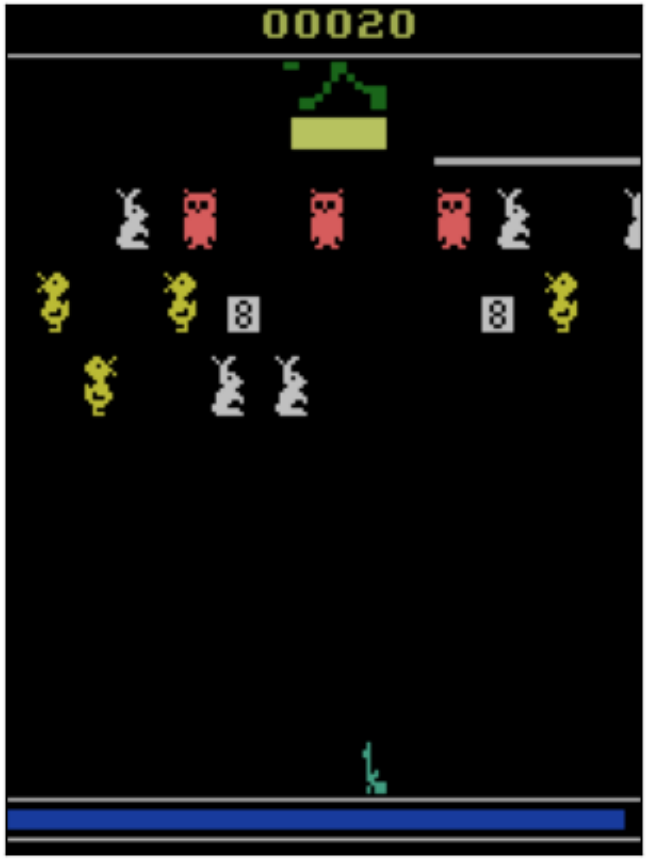


Teacher: Fire

Student: Move

Training trajectory

***t* = 2**



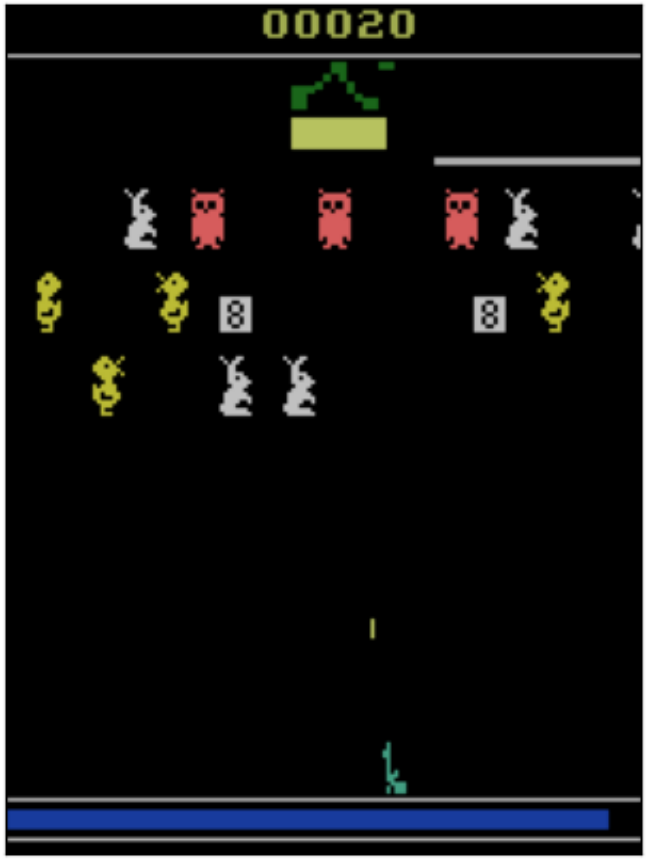
Teacher: Fire

Student: Move

…such as firing at enemy targets

Training trajectory

$t = 3$

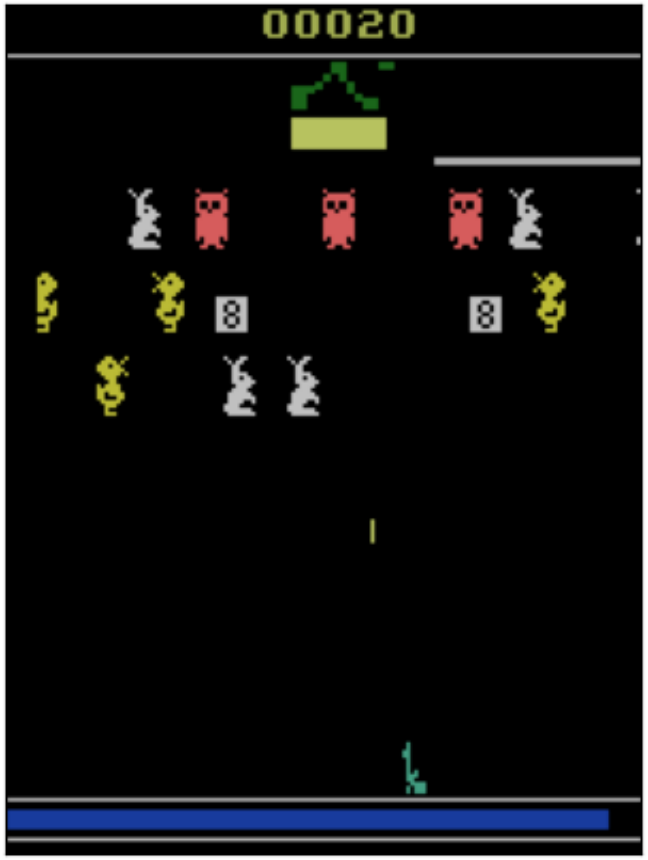


Teacher: Move

Student: Fire

Training trajectory

$t = 4$

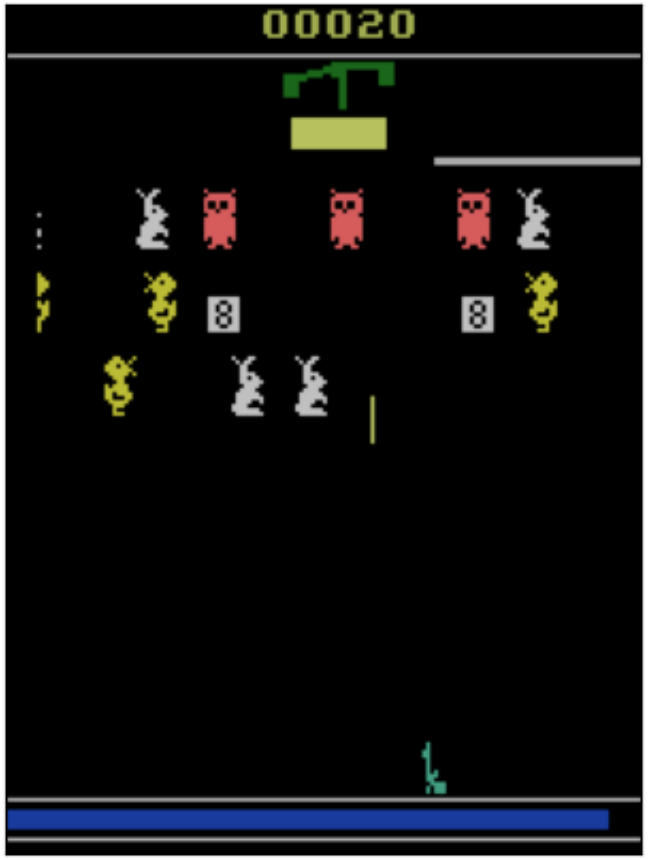


Teacher: Move

Student: Move

Training trajectory

$t = 5$

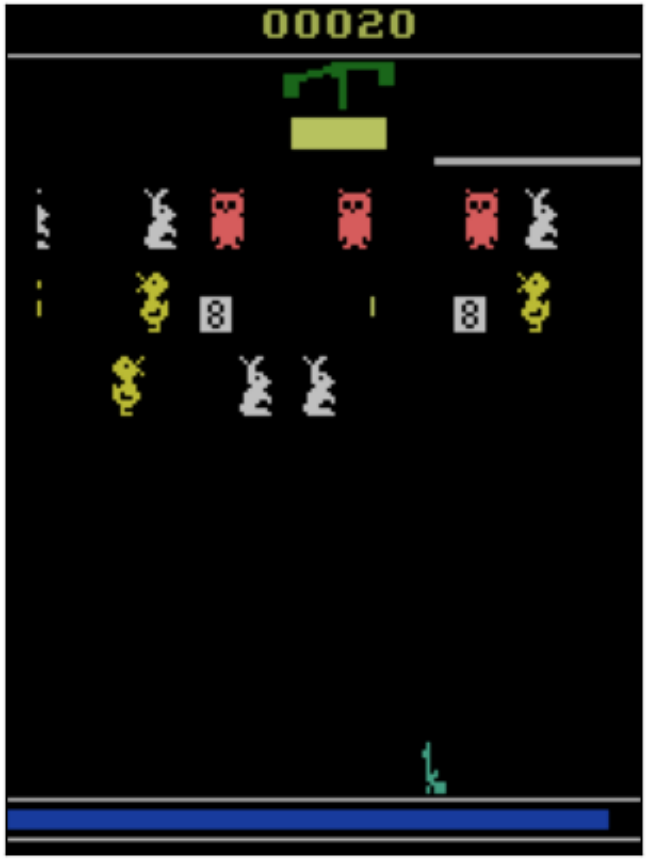


Teacher: Move

Student: Move

Training trajectory

$t = 6$

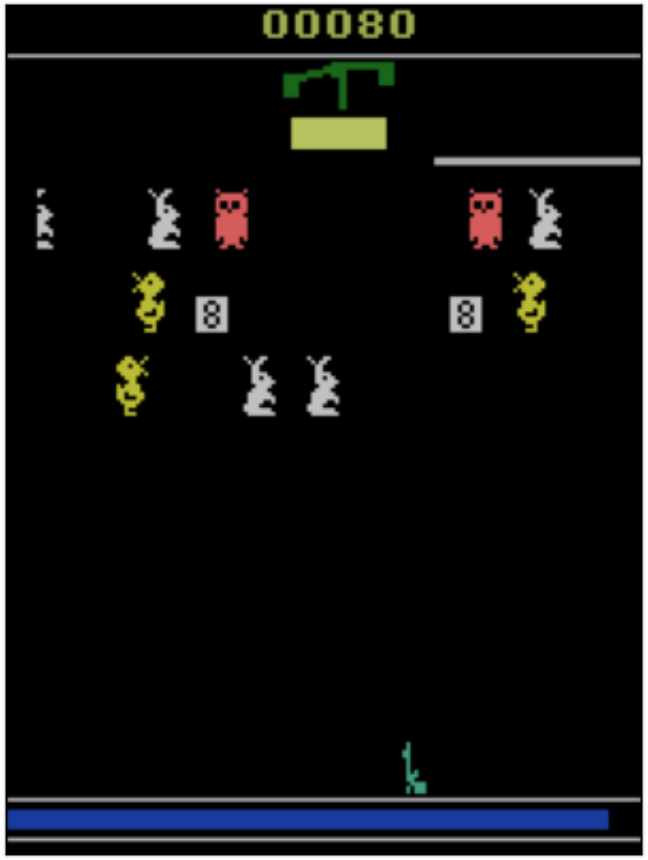


Teacher: Move

Student: Move

Training trajectory

***t* = 7**

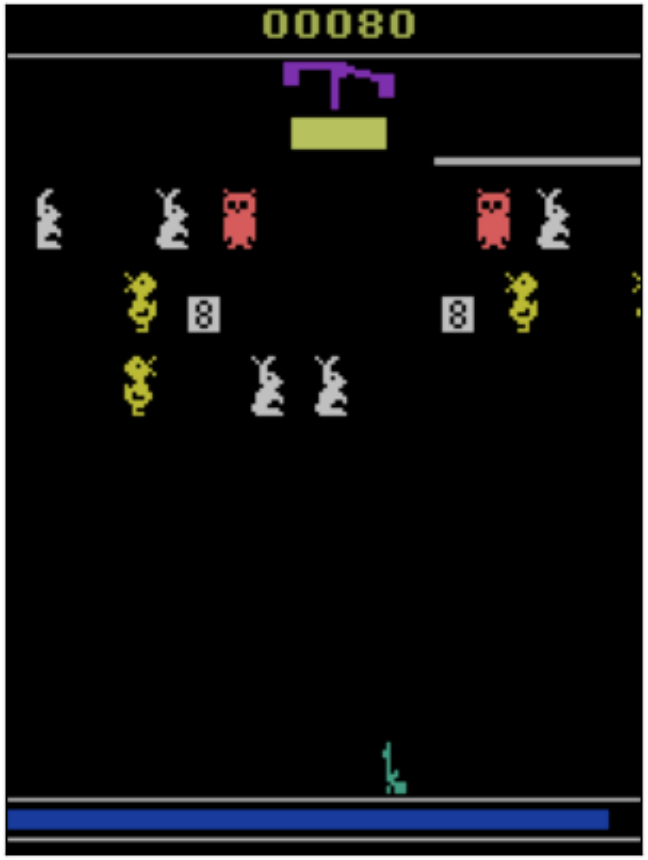


Teacher: Move

Student: Move

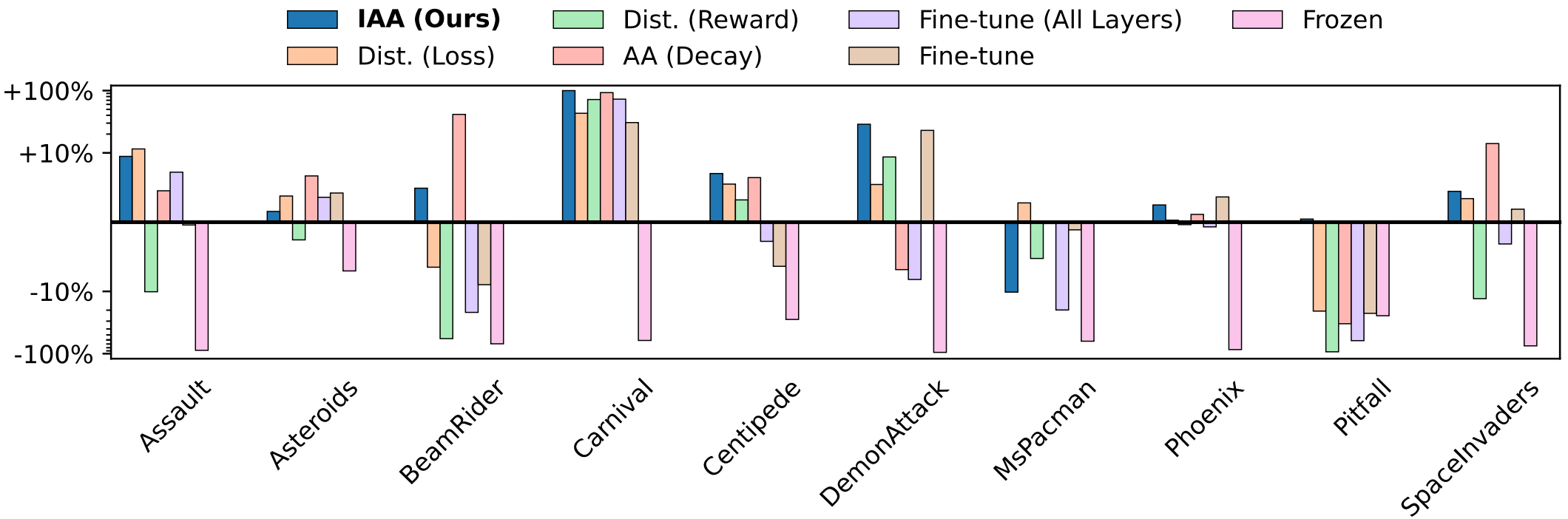
Training trajectory

$t = 8$



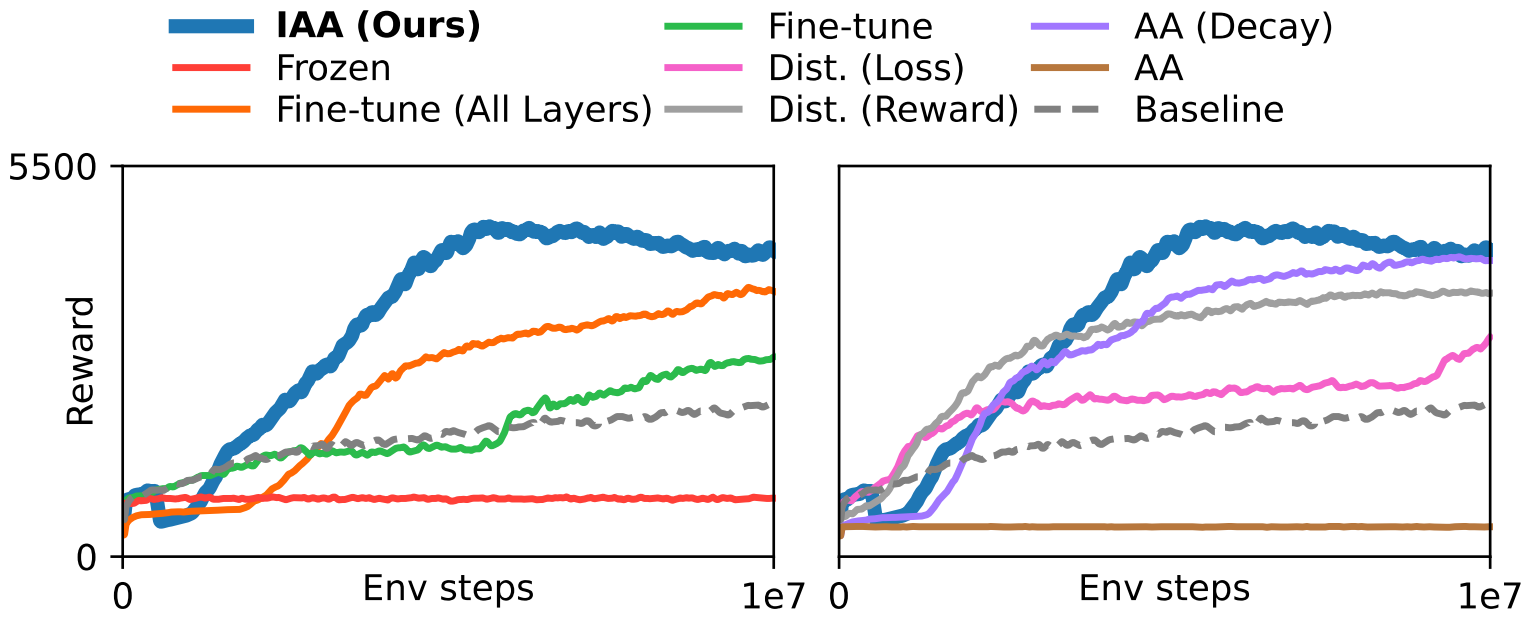
Teacher: Move

Student: Move

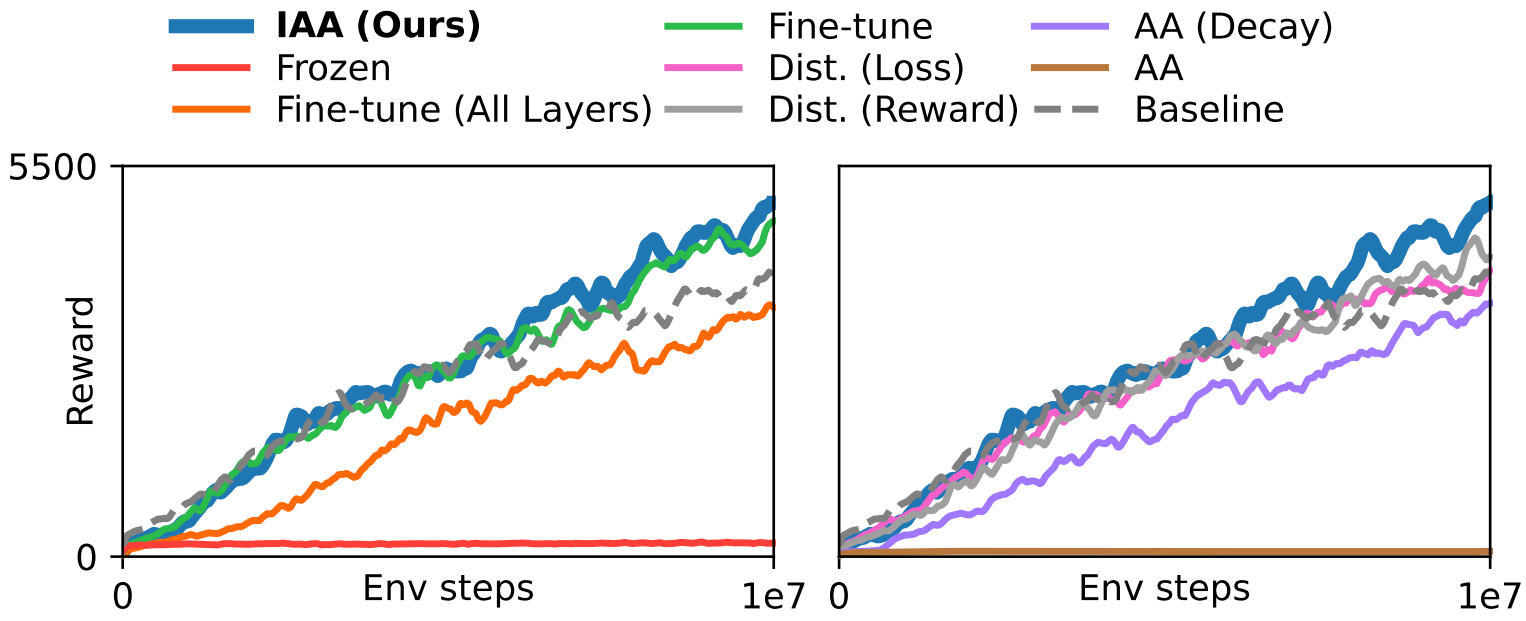


Metric	**IAA (Ours)**	Dist. (Loss)	Dist. (Reward)	AA (Decay)	Fine-tune (All Layers)	Fine-tune	Frozen
Positive Transfer Ratio	**9/10**	8/10	3/10	7/10	3/10	5/10	0/10
Minimum Transfer	**–10%**	–21%	–93%	–48%	–90%	–33%	–95%
Median Transfer	4.6%	3.6%	–3.9%	**5.4%**	–3.0%	0.7%	–66%
Maximum Transfer	**99%**	43%	72%	93%	73%	31%	–7%

Atari – Carnival



Atari – Demon Attack



Take-aways

Intelligent exploration is crucial for overcoming sparse rewards and temporally extended tasks

Intrinsic motivation: agent creates its own rewards

- Novelty-seeking and modeling error approaches

Extrinsic motivation: agent receives external reward

- One possible type: teacher agent helps guide student